

Vorlesung Cybersicherheit, Sommersemester 2025

Software Exploits

Prof. Dr.-Ing. Lucas Vincenzo Davi

Fakultät für Informatik

Arbeitsgruppe Systemsicherheit <https://www.syssec.wiwi.uni-due.de/>

Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi

- Rückblick
 - TLS
 - Denial-of-Service
 - Input Validation
- Themen heute und nächste Woche
 - Software Exploits
 - Code Injection
 - Code-Reuse
 - CFI und ASLR
 - Data-Oriented Exploits



Kryptografie

- Symmetrische Kryptografie
- Asymmetrische Kryptografie
- Hash Funktionen und Digitale Signaturen
- Sicherheitsprotokolle
- Kryptowährungen

Netzwerk Sicherheit

- Wireless Security
- Netzwerkangriffe
- Web Sicherheit

Software und Systemsicherheit

- Software Exploits
- Malware
- Betriebssystemssicherheit
- Confidential Computing

Motivation

- Software increasingly sophisticated and complex
- Software needs to be developed for different computer architectures
- Many developers without any security background
- Native Code C/C++



High probability for program bugs

Motivating C Code Examples

based on "Memory corruption - public enemy number 1", Erik Poll, Digital Security Radboud University Nijmegen
https://www.cs.ru.nl/~erikpoll/ss/2019/slides/2_MemoryCorruption.pdf

A simple example in C

Array declaration in C

- `char buffer[8];`

Bad access

- `buffer[8] = „A“`

Undefined program behavior

- Segmentation fault
- Arbitrary Code Execution

Compiler

- Maybe the compiler catches the bug, but if the array field depends on runtime data it could miss the bug, e.g.,
`buffer[index] = „A“`

The Problem of Dangerous String Functions in C

```
#include <string.h>
```

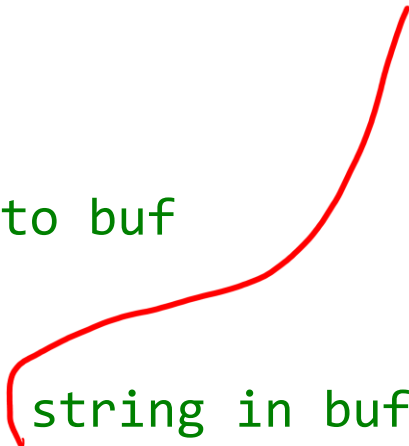
```
void function() {  
    char buf[20];  
    char prefix[] = "http://";  
    char *path;
```

```
    // Copy the string prefix to buf  
    strcpy(buf, prefix);
```

```
    // Concatenate path to the string in buf, ensuring size limit  
    strncat(buf, path, sizeof(buf));
```

```
}
```

20 7
sizeof(buf) - strlen(buf)
- 1
↓
10



Handwritten Notes for Code Listed in the Previous Slide

<code>buf = http://</code> 7 Byte	<code>www.uni-due.de</code> 13 Byte	Variable
--------------------------------------	--	----------

`prefix[] = "http://"`

Null Byte

`http://\0`

Another Buggy String Example

```
#include <string.h>
```

```
void function() {  
    char src[14];  
    char dest[14];  
    char *base_url = "www.uni-due.de";
```

```
    // Copy base_url to src with a limit of 14 characters  
    strncpy(src, base_url, sizeof(src));
```

```
    // Copy src to dest  
    strcpy(dest, src);
```

```
}
```

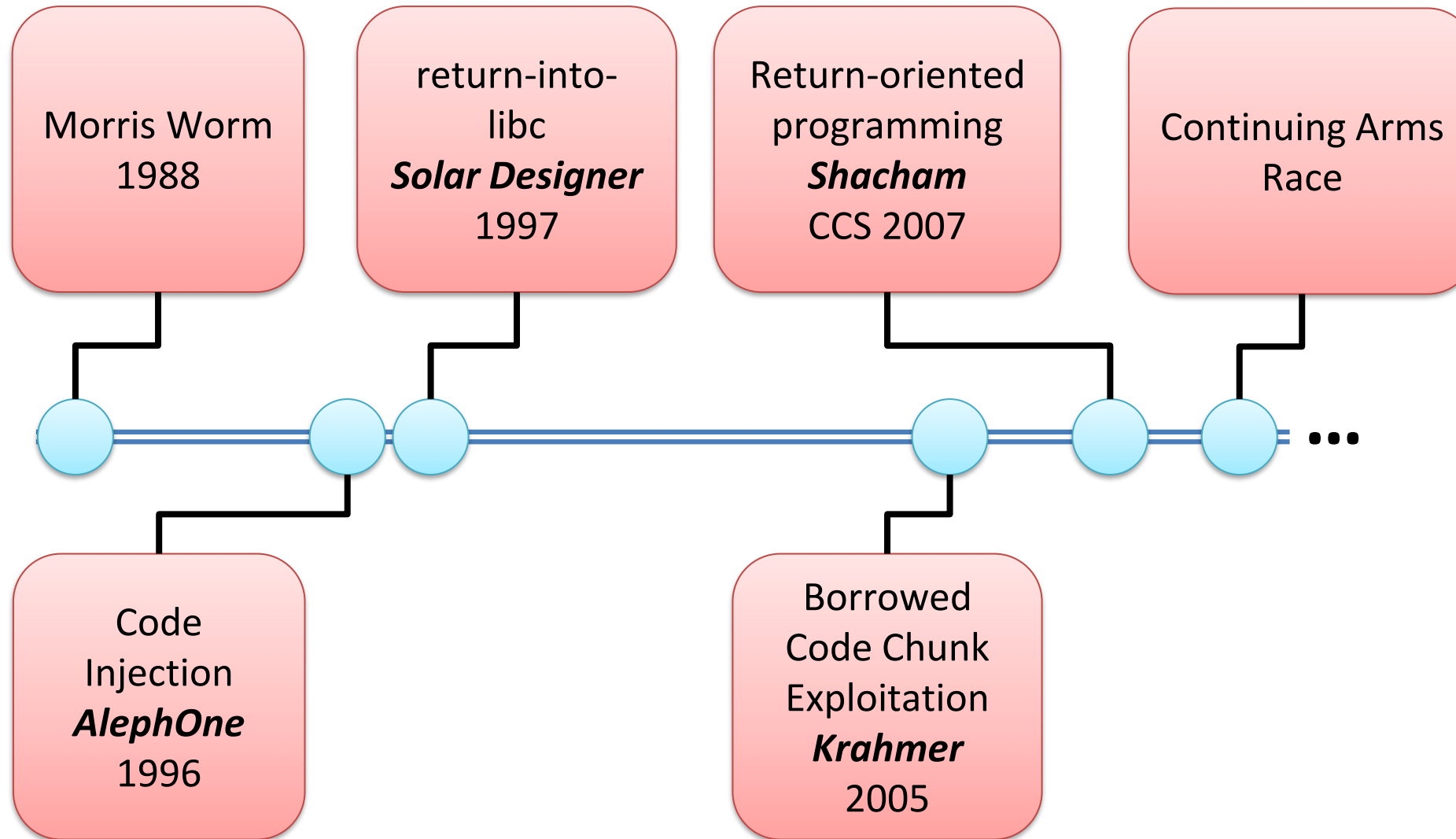
1 2 3
src = www . . .

14
e | A B C D | 0

dest = www.uni-due.de a b c d | 0

Exploitation of Software Vulnerabilities

Four Decades of Run-Time Attacks



High Impact of Attacks

- Web browsers repeatedly exploited in pwn2own contests
- Zero-day issues exploited in Stuxnet/Duqu

Industry Efforts on Defenses

- Microsoft EMET (Enhanced Mitigation Experience Toolkit) includes a ROP detection engine
- Microsoft Control Flow Guard (CFG) in Windows 10
- Google's compiler extension VTV (virtual table verification)
- Intel Control-flow Enforcement Technology (CET)
- Kernel CFI (kCFI) since Android 9

But software exploits have also some “good” side-effects



Apple iPhone Jailbreak

Disable signature verification and escalate privileges to root



Request

http://www.jailbreakme.com/_/iPhone3,1_4.0.pdf



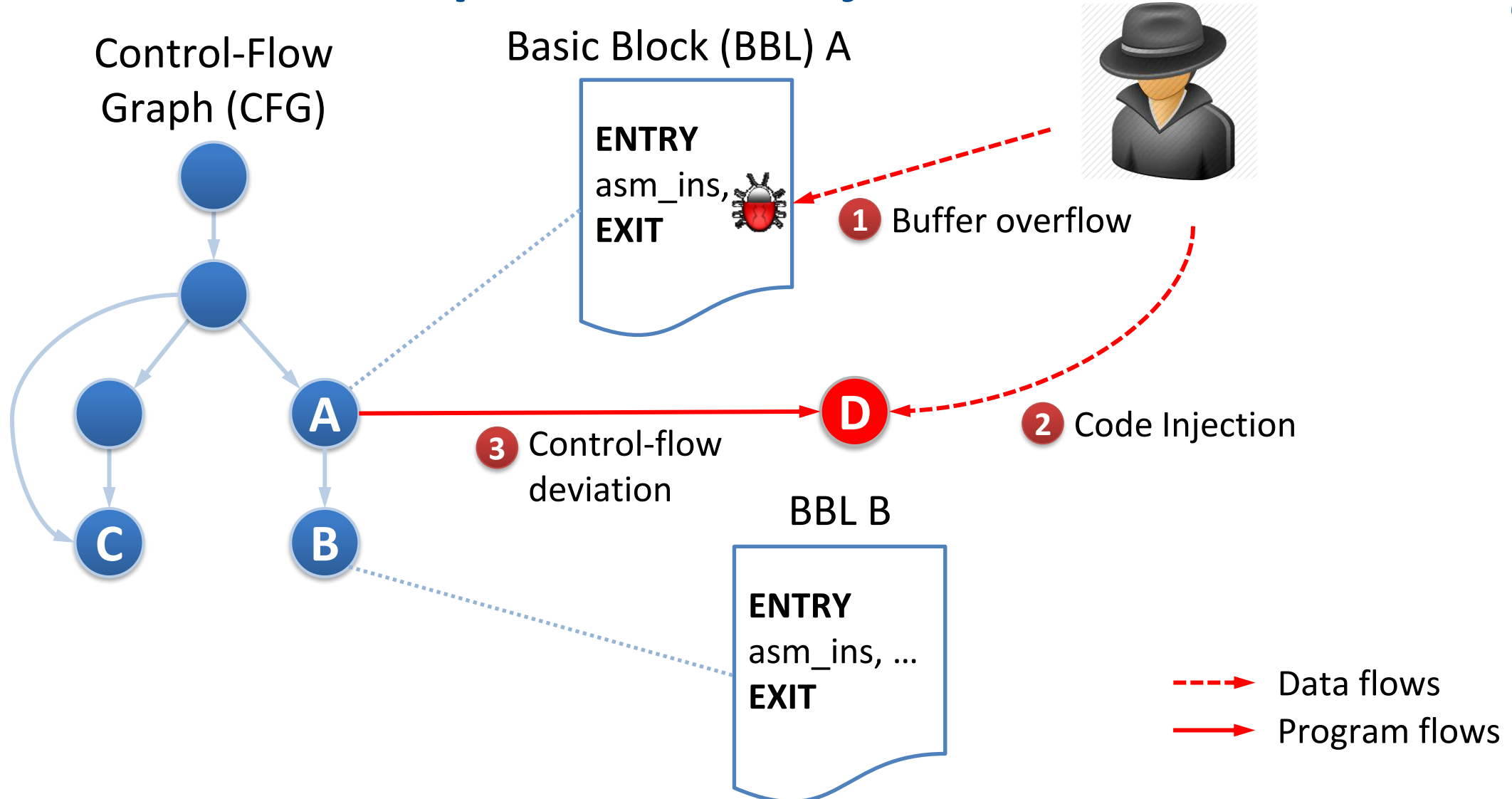
- 1) Exploit PDF Viewer Vulnerability by means of **Return-Oriented Programming**
- 2) Start Jailbreak
- 3) Download required system files
- 4) Jailbreak Done

How can an attacker exploit vulnerabilities in a meaningful way?

Code Injection
Attacks

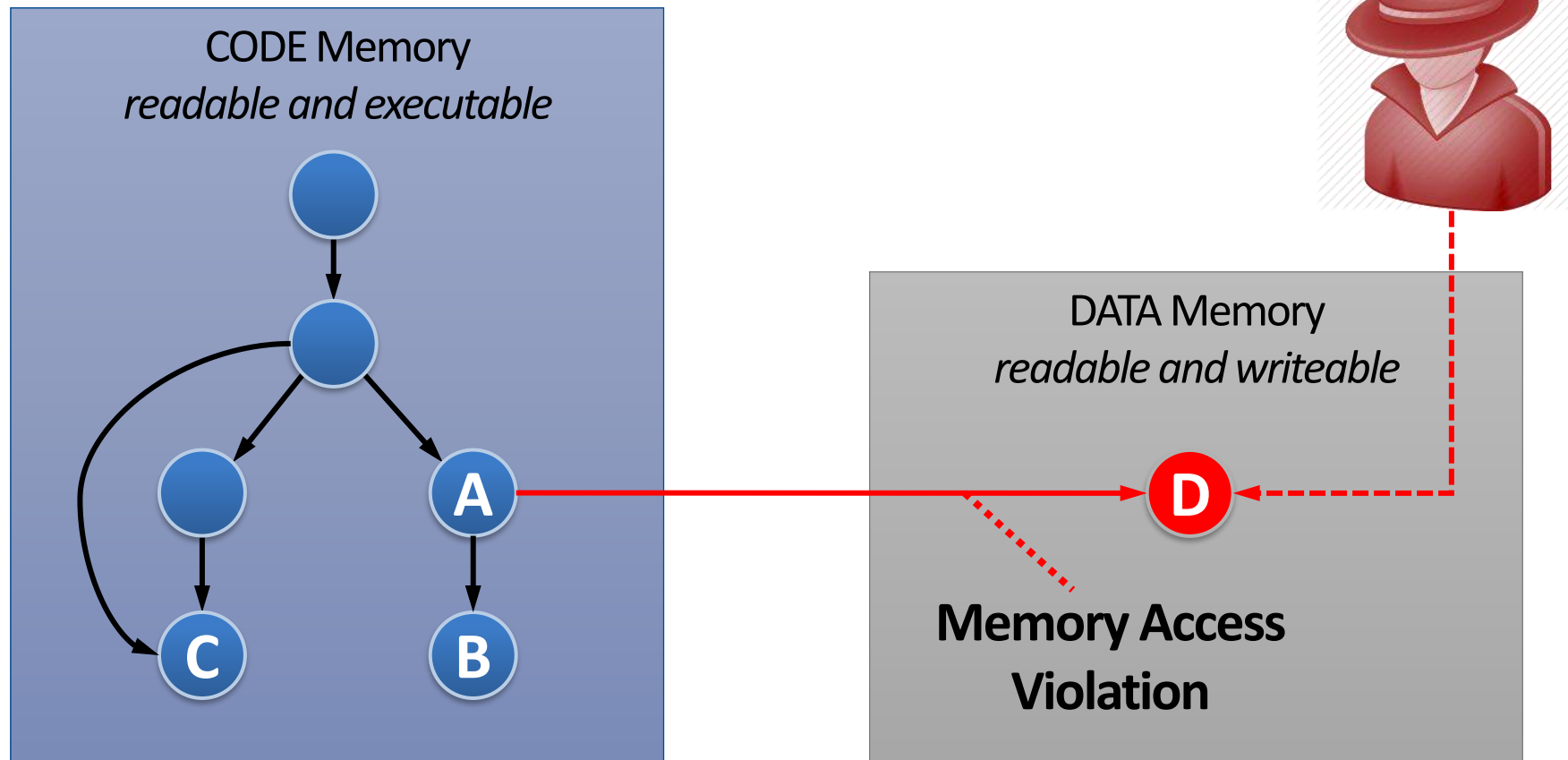
Code-Reuse
Attacks

General Principle of Code Injection Attacks

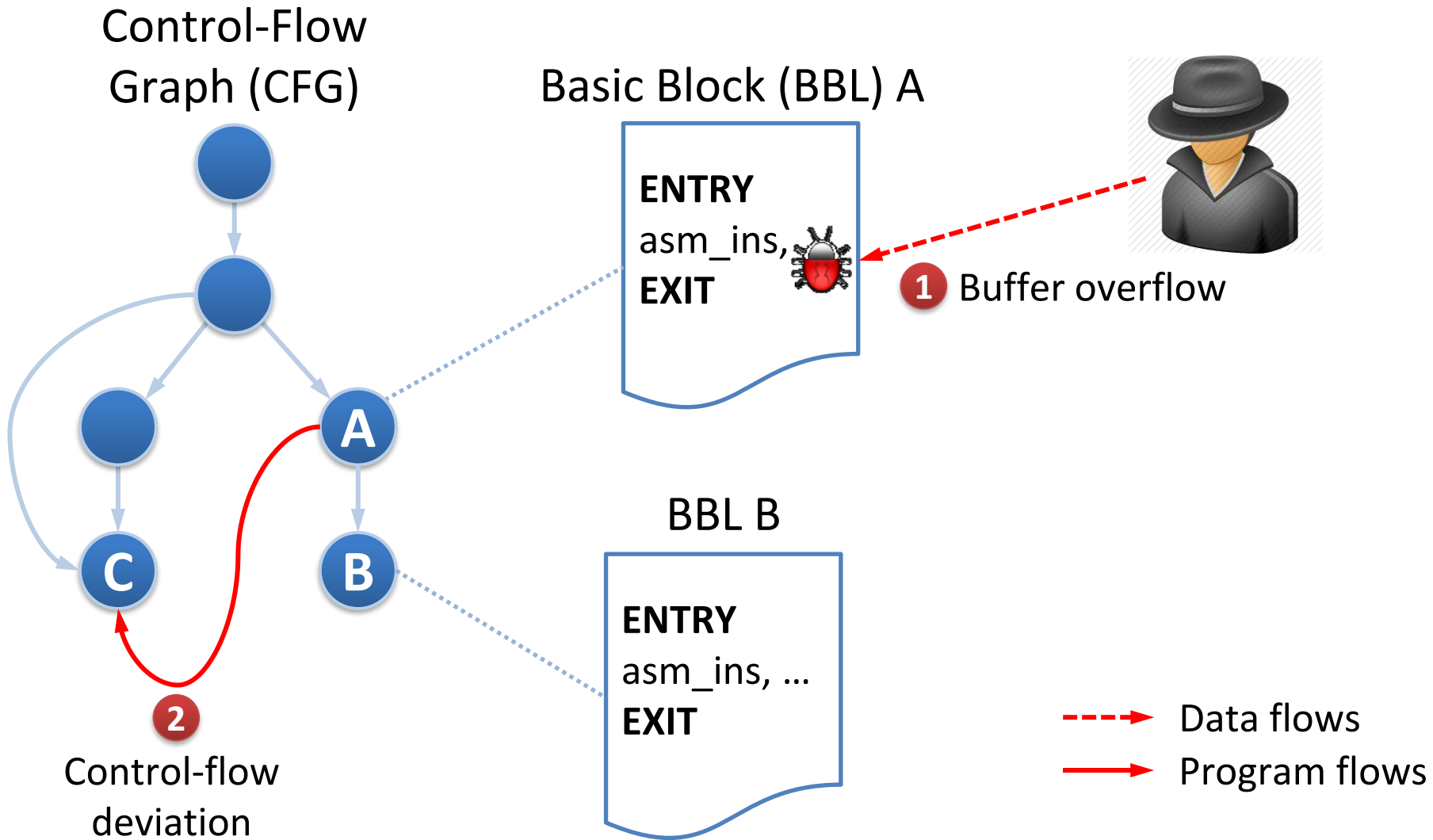


Data Execution Prevention (DEP)

Prevent execution from a writeable memory (data) area



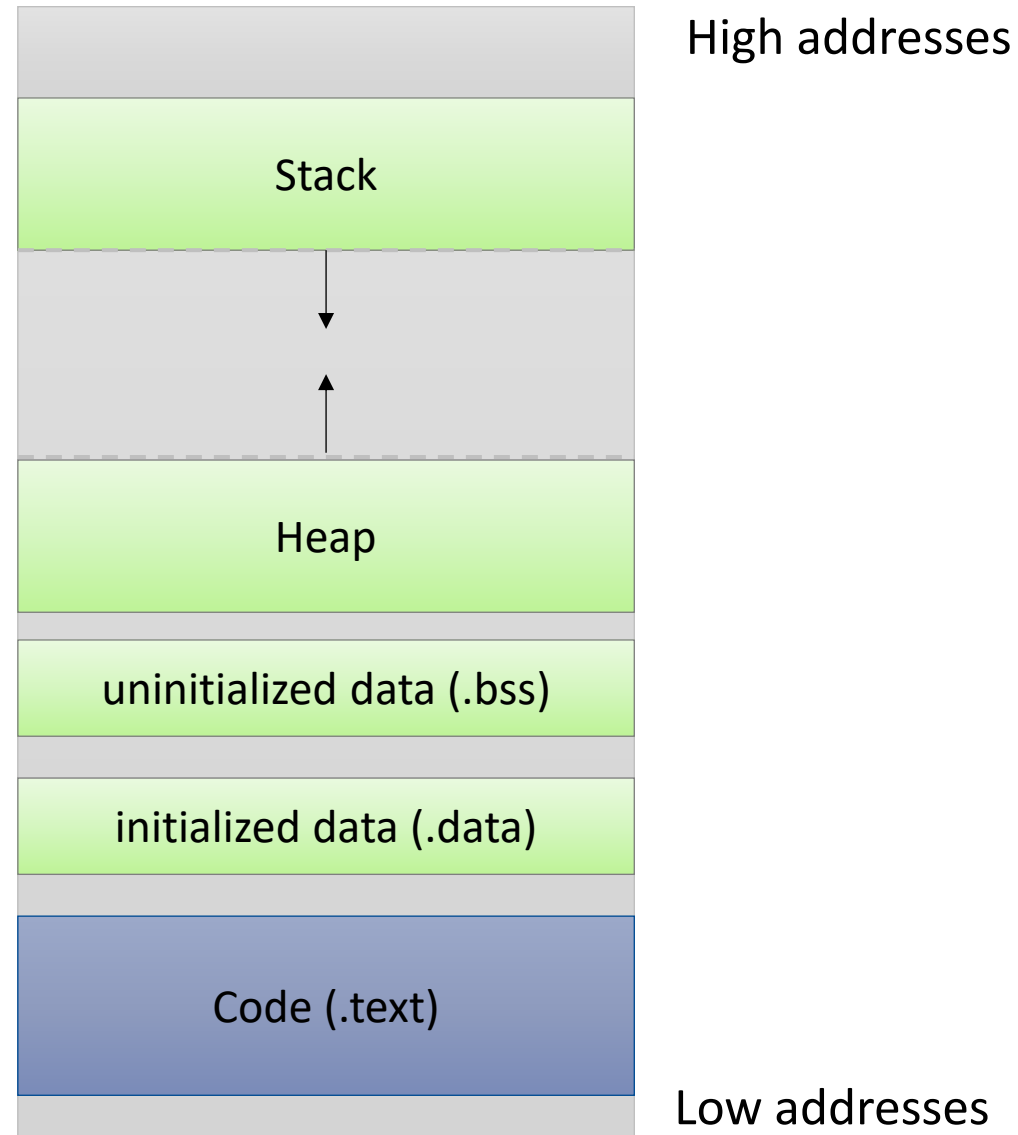
General Principle of Code-Reuse Attacks



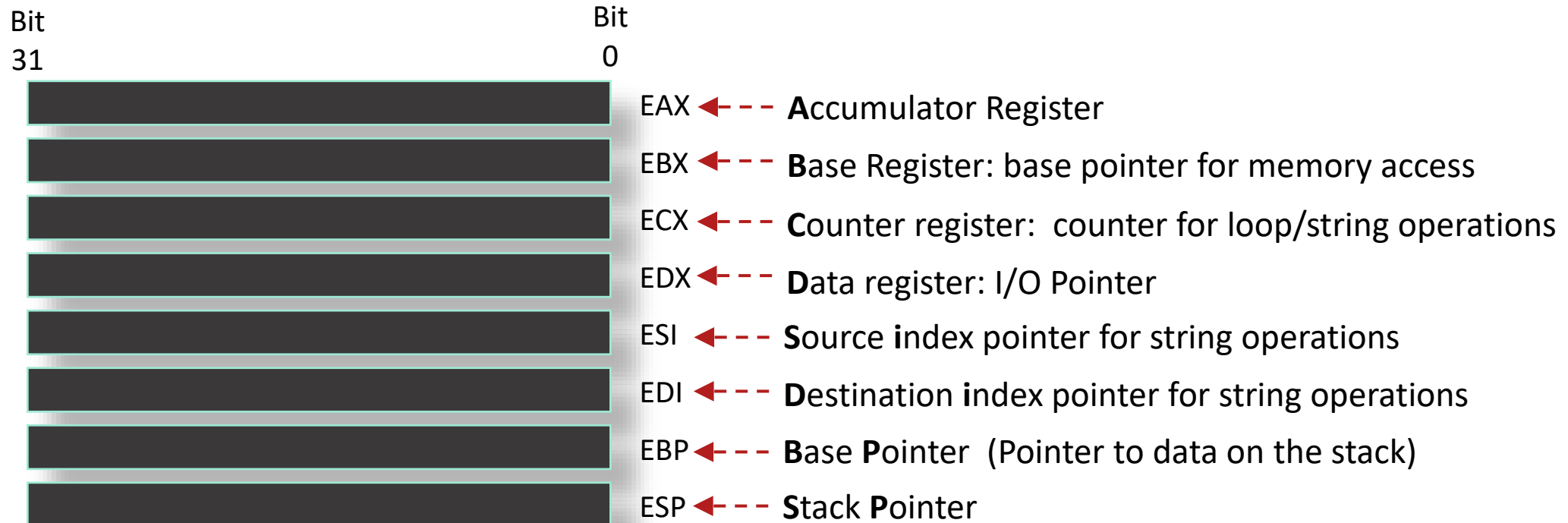
- Code Injection – *Adding a new **node** to the CFG*
 - Adversary can execute arbitrary malicious code
 - open a remote console (classical shellcode)
 - exploit further vulnerabilities in the OS kernel to install a virus or a backdoor
- Code Reuse – *Adding a new **path** to the CFG*
 - Adversary is limited to the code nodes that are available within the address space
 - Requires reverse-engineering and static analysis of the code base of a program

Before we describe the implementation details of the attack, we briefly introduce some basic x86 mechanisms

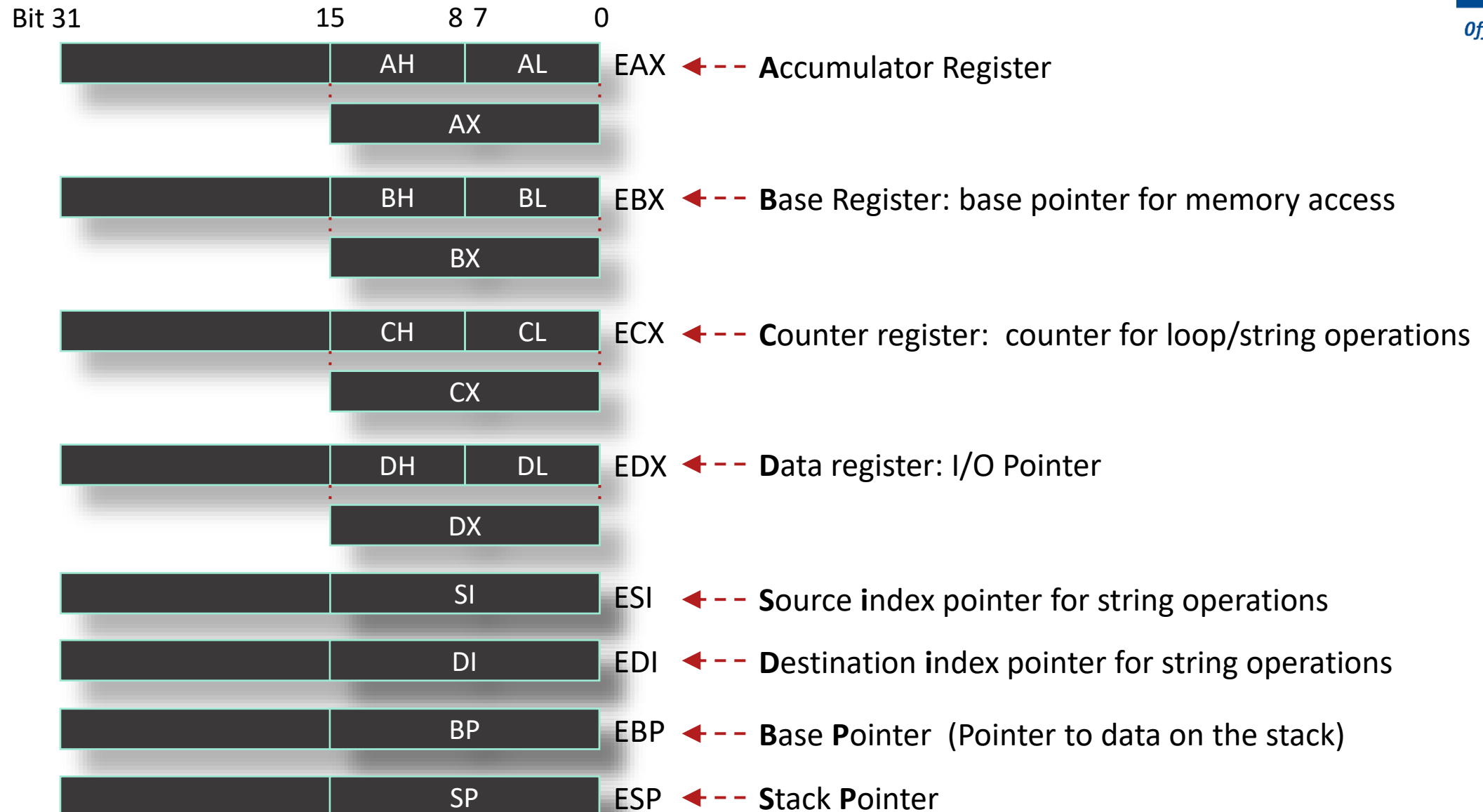
Simplified Linux C Process Layout



Eight General-Purpose Registers on x86

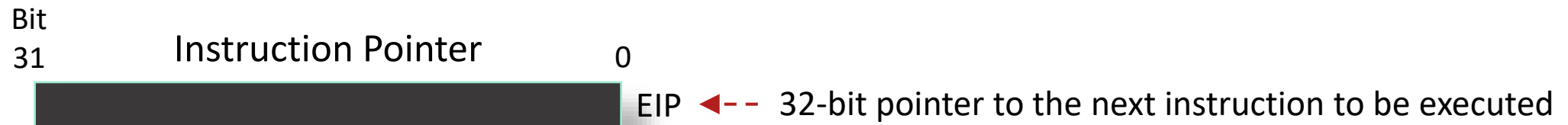


8- and 16-Bit General-Purpose Registers



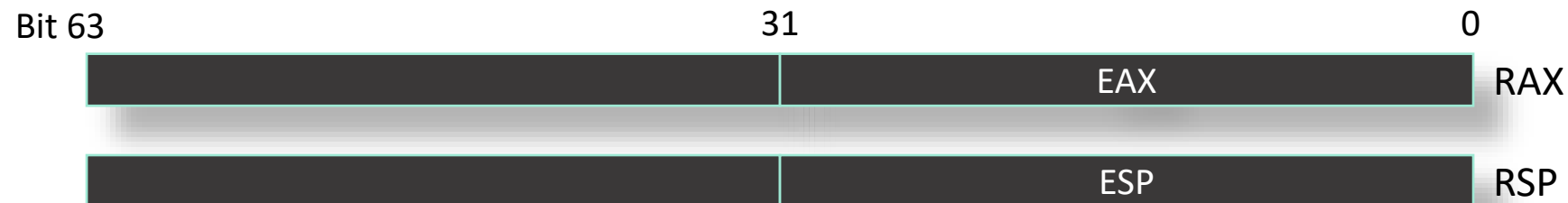
Instruction Pointer Register

- The instruction pointer (EIP) points to the instruction that should be executed next
- EIP is not a general-purpose register, i.e., it cannot be accessed by any instruction except explicit branch instructions such as **CALL, JMP, RET**

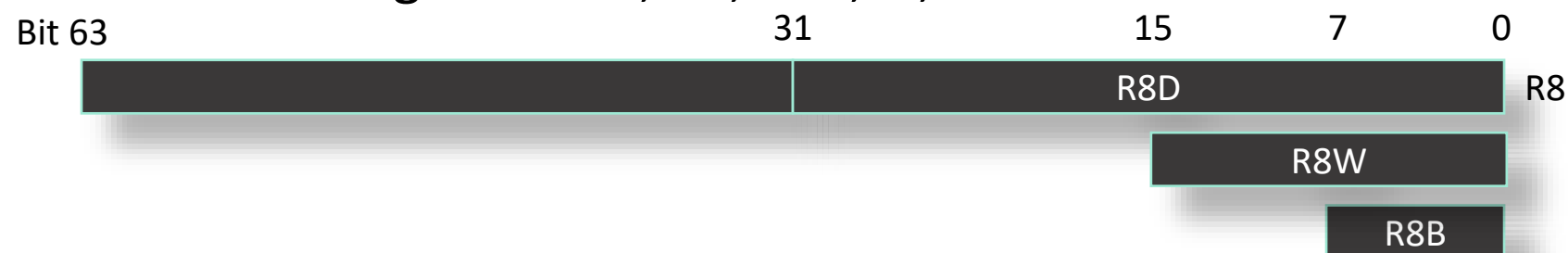


x86_64 Registers

- General-purpose registers are extended to 64 Bit registers, e.g.,
 - EAX->RAX
 - EBX ->RBX
 - ...
 - EIP->RIP
 - EFLAGS->RFLAGS



- Additional registers: R8, R9, R10, ..., R15



- The MOV instruction is utilized to perform a data movement operation on x86



- Different types of data movement available
 1. Immediate to register
 2. Register to register
 3. Immediate to memory
 4. Register to memory / memory to register

Beispiel für mov Instruktion

dst src
↓ ↓

mov eax, 15 : eax = 15

mov ebx, eax : ebx = 15

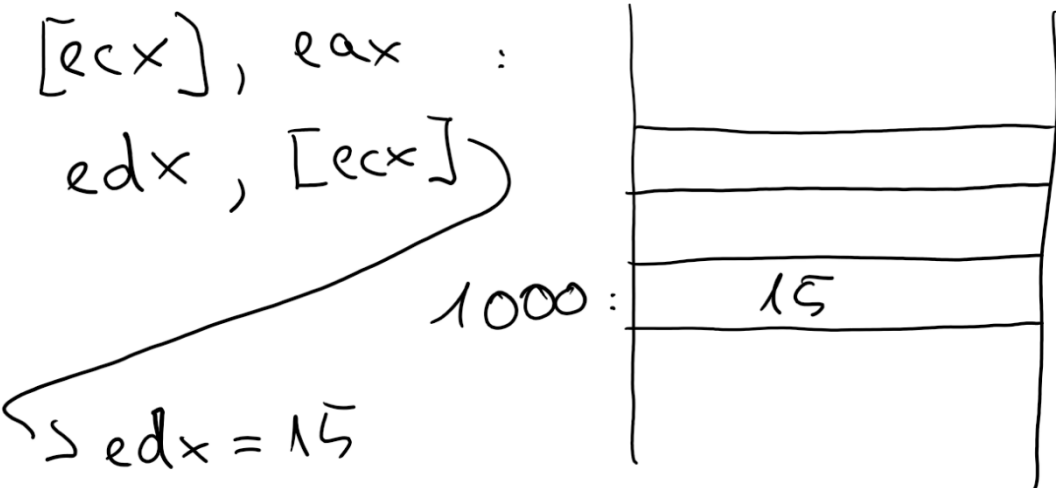
mov ecx, 1000 : ecx = 1000

mov [ecx], eax :

mov edx, [ecx]

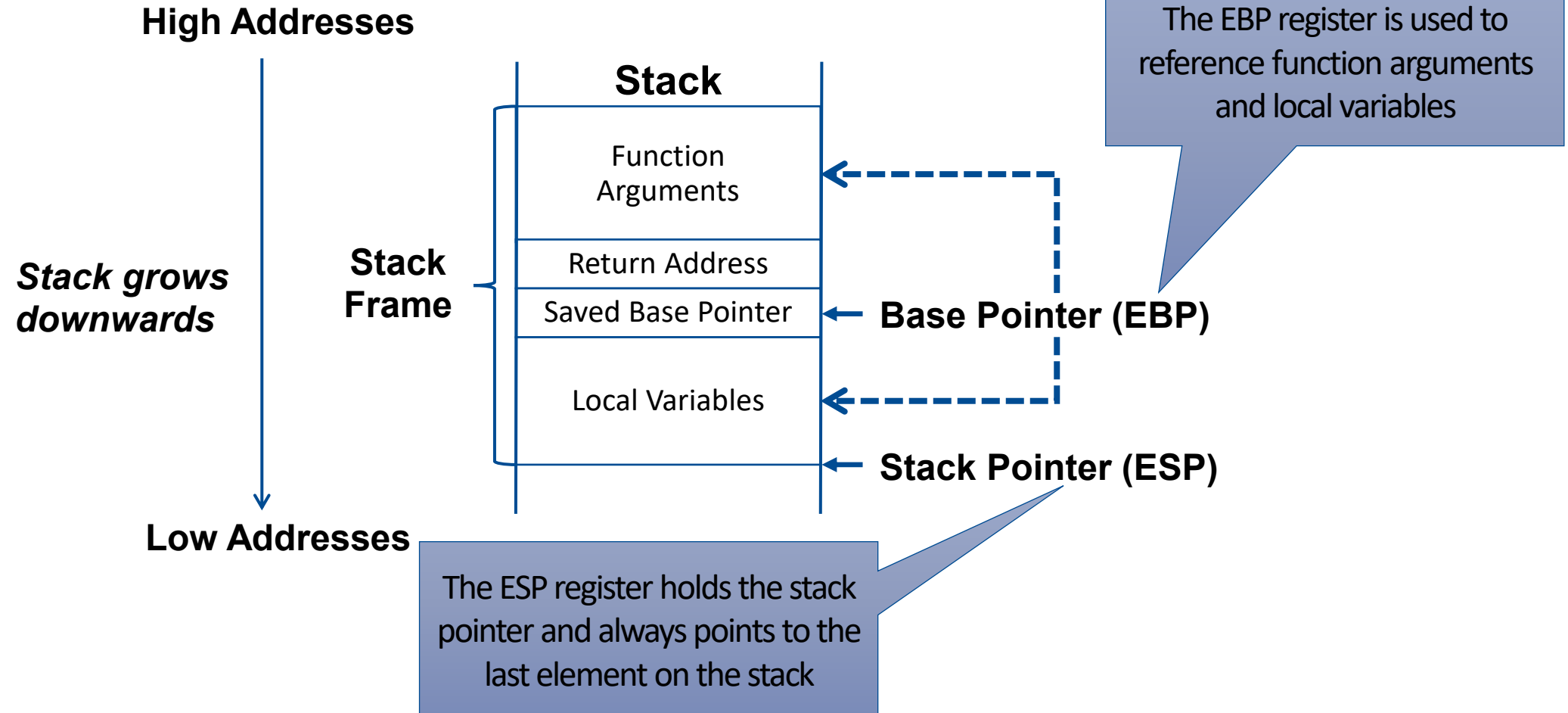
1000: 15

edx = 15



Stack Frame

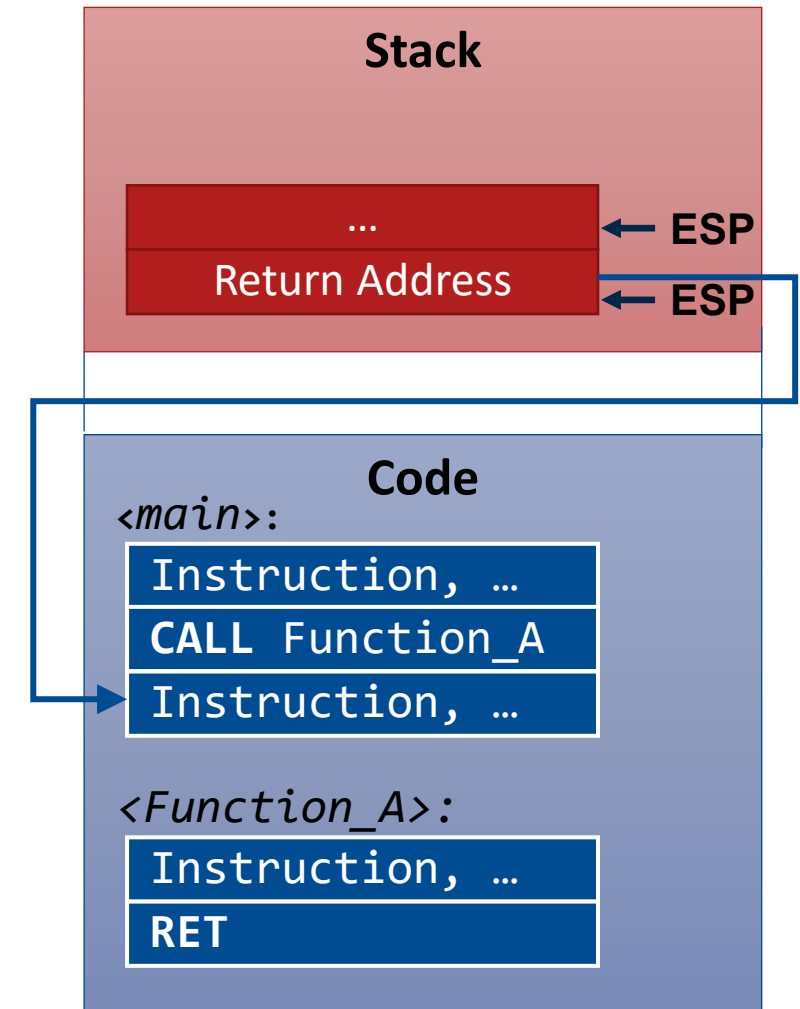
Each function is associated with one stack frame on the stack



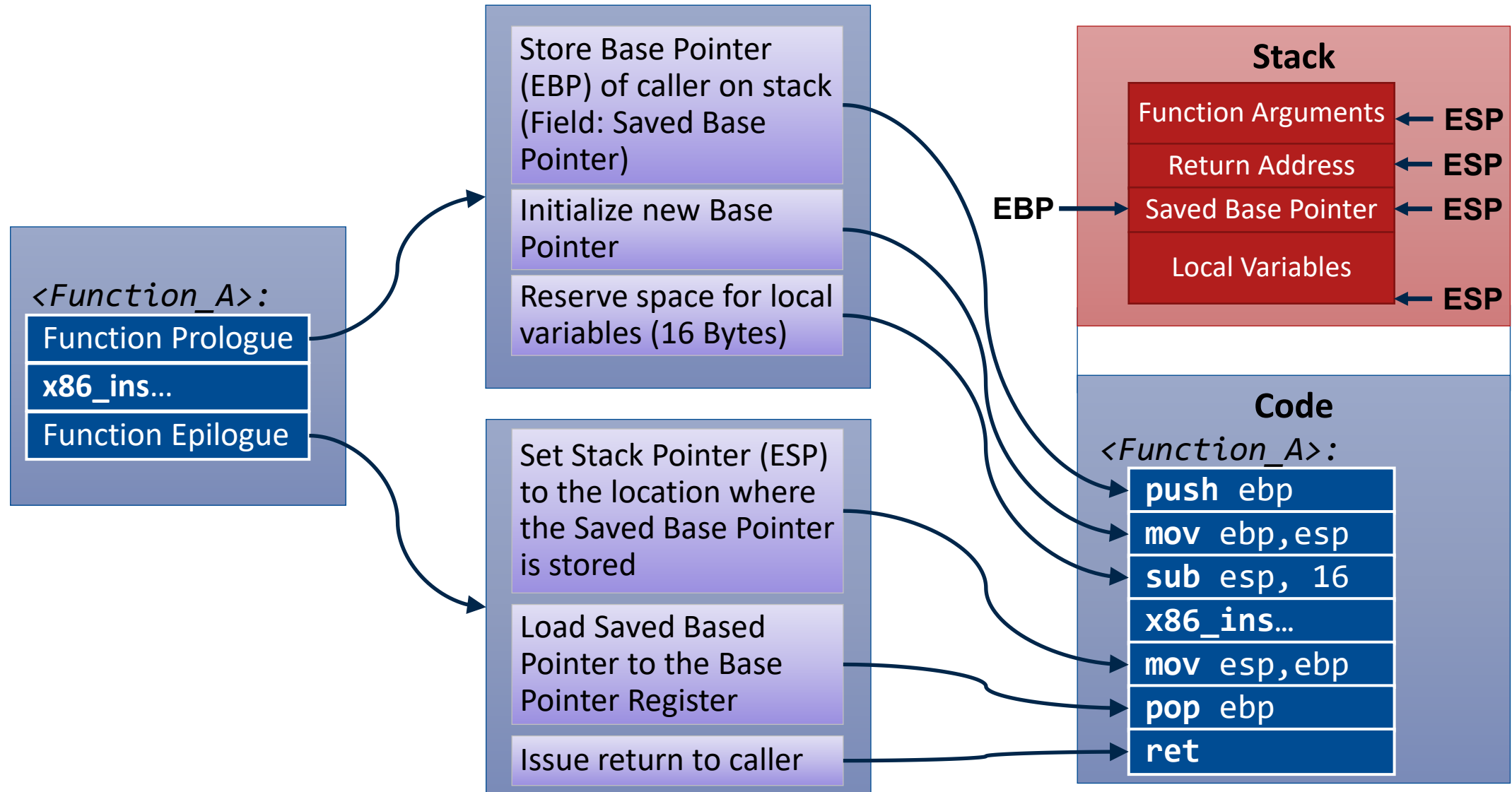
- Stack is divided into individual stack frames
 - Each function call sets up a new stack frame on top of the stack
 1. **Function arguments**
 - Arguments provided by the caller of the function
 2. **Return address**
 - Upon function return (i.e., a return instruction is issued), control transfers to the code pointed to by the return address (i.e., control transfers back to the caller of the function)
 3. **Saved Base Pointer**
 - Base pointer of the calling function
 4. **Local variables**
 - Variables that the called function uses internally

Calling Convention (on Intel x86)

- Function call performed via the x86 **call** instruction
 - E.g., **call** *Function_A*
 - The **call** instruction automatically pushes the return address on the stack, while the return address simply points to the instruction preceding the call
- Function return is performed via the x86 **ret** instruction
 - The **ret** instruction pops the return address off the stack and loads it into the instruction pointer (EIP)
 - Hence, the execution will continue in the main function



Function Prologue and Epilogue by Example



Implementation of a code injection attack

Example

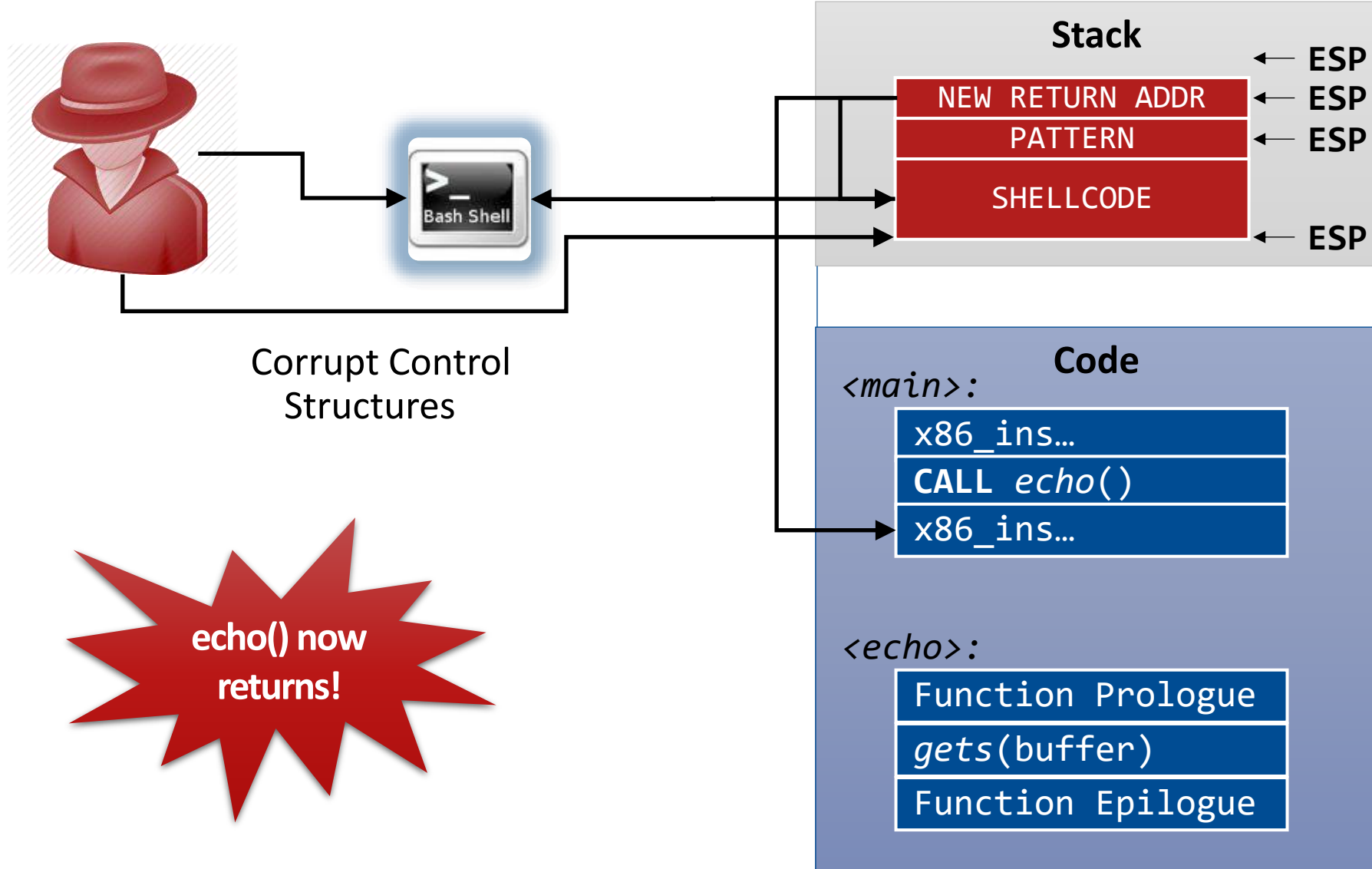
Simple Echo program suffering from a stack overflow vulnerability

- `gets()` expects an input from the command line
- `puts(buffer)` outputs the string of the buffer array on the command line

The `gets()` function does not provide bounds checking

```
#include <stdio.h>
void echo()
{
    char buffer[80];
    gets(buffer);
    puts(buffer);
}
int main ()
{
    echo();
    printf("Done");
    return 0;
}
```

Example: Code Injection Attack



However, code injection can be prevented by data execution prevention

return-into-libc attacks

[Solar Designer, 1997]

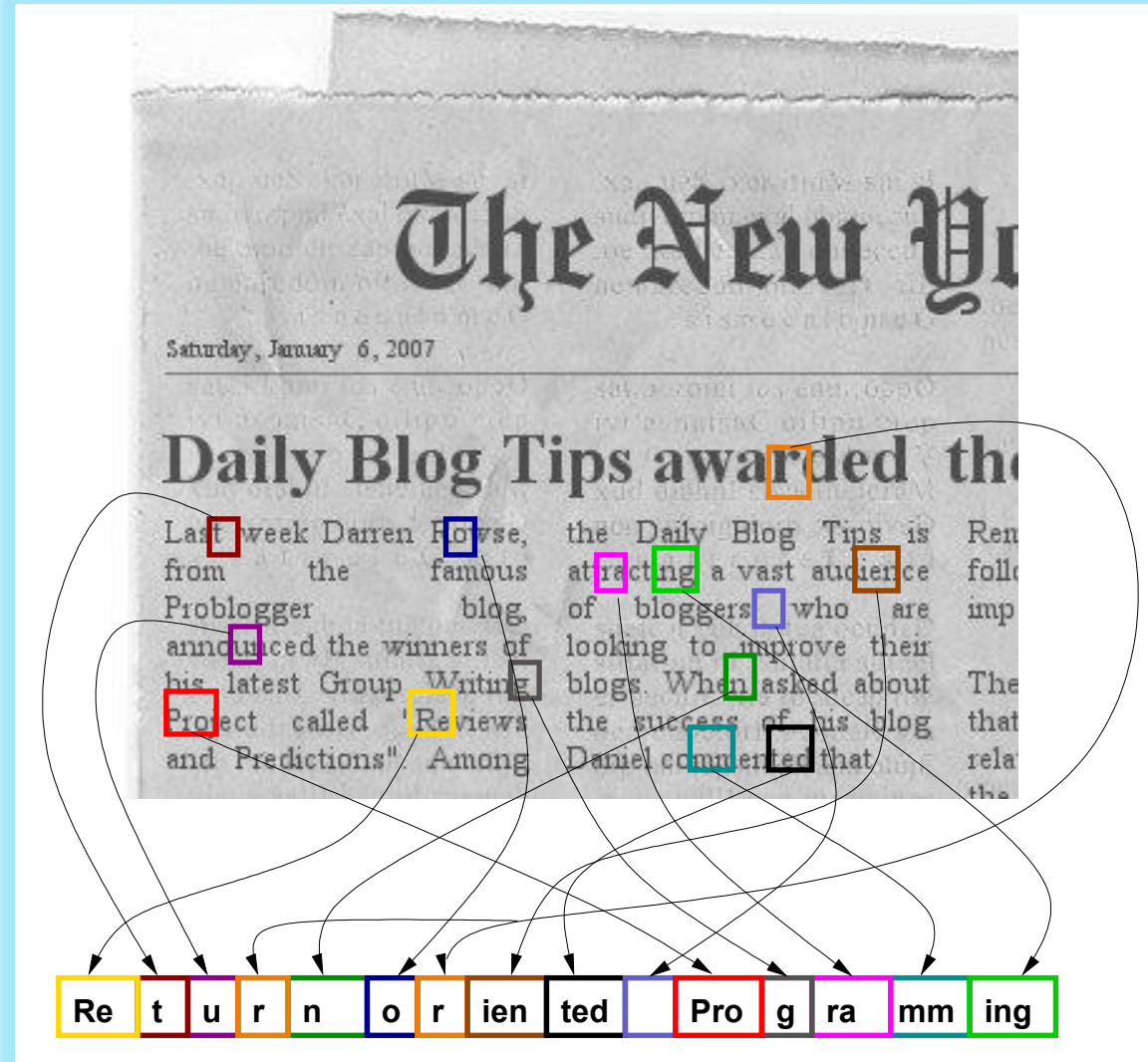
- Basic idea
 - Redirect execution to security-critical functions in shared libraries
 - Main target of original attack was the UNIX C library libc
 - libc is linked to nearly every Unix program
 - Features several operating system services such as *open()*, *malloc()*, *printf()*, *system()*, *execve()*, etc.
 - Attack example
 - `system("/bin/sh"), exit()`

Limitations of return-into-libc Attacks

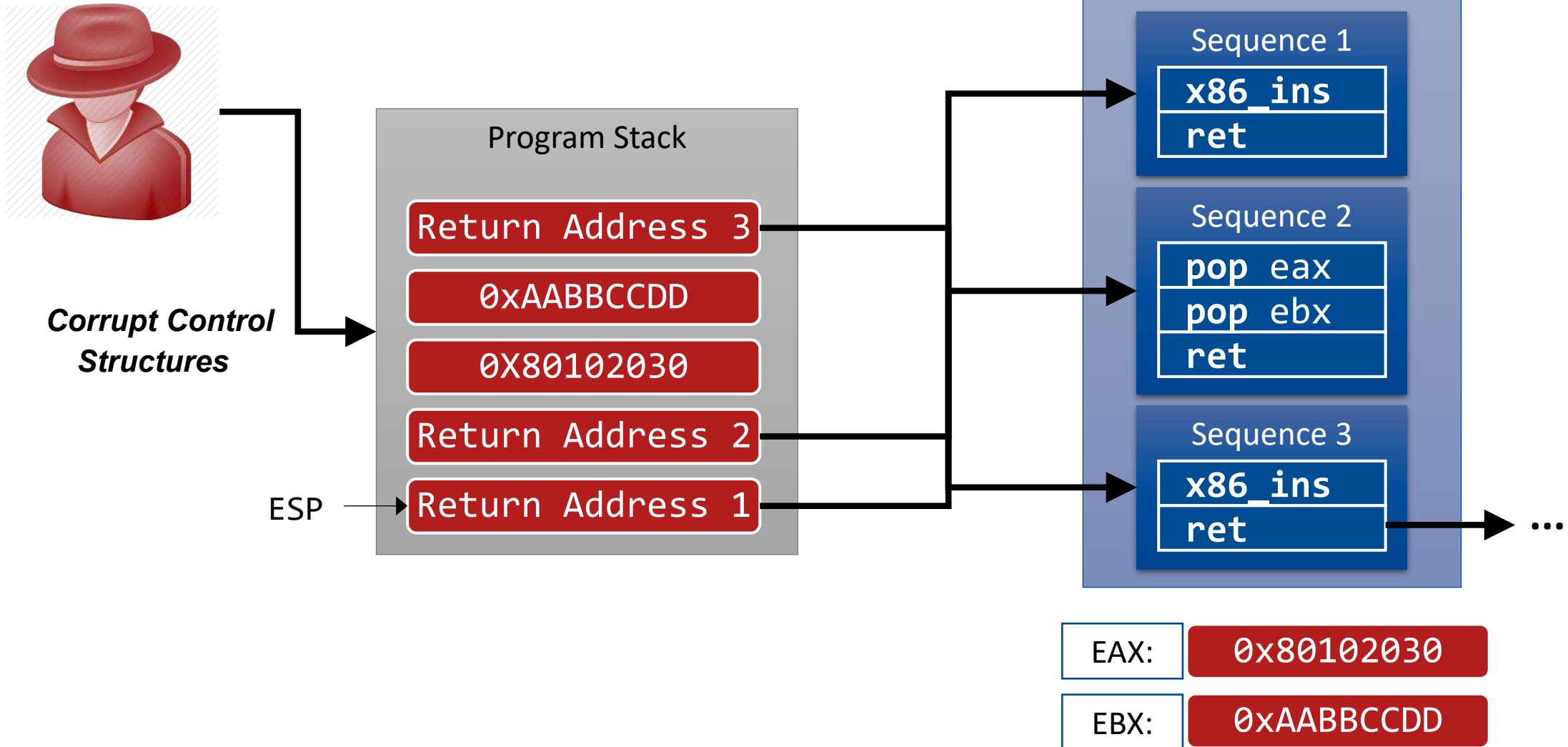
- The adversary relies on functions available in `libc`. Hence, the designers of `libc` could eliminate or add secure wrappers of critical functions such as *system()*.
- The adversary can only invoke one function after the other; not supporting conditional branching. Implementing arbitrary malicious computation is not directly possible.

Generalization of return-into-libc attacks: return-oriented programming [Shacham, ACM CCS 2007]

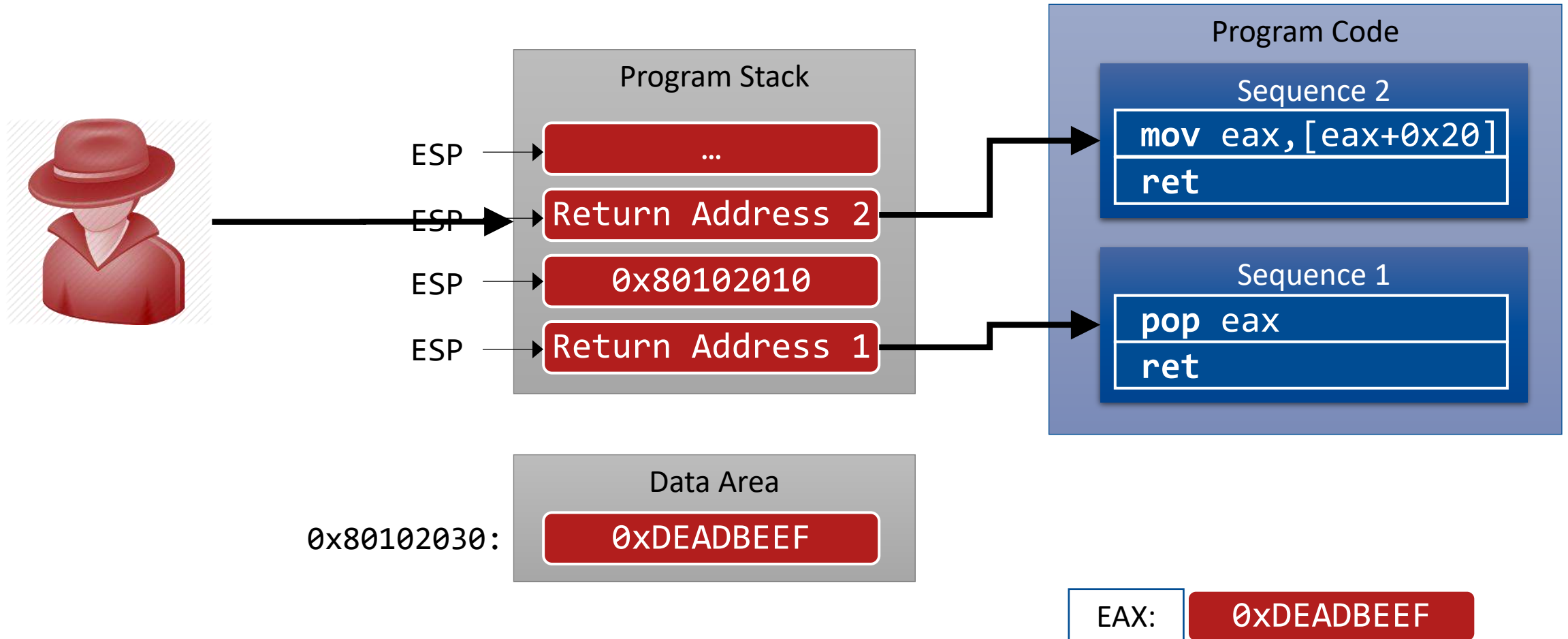
Big Picture



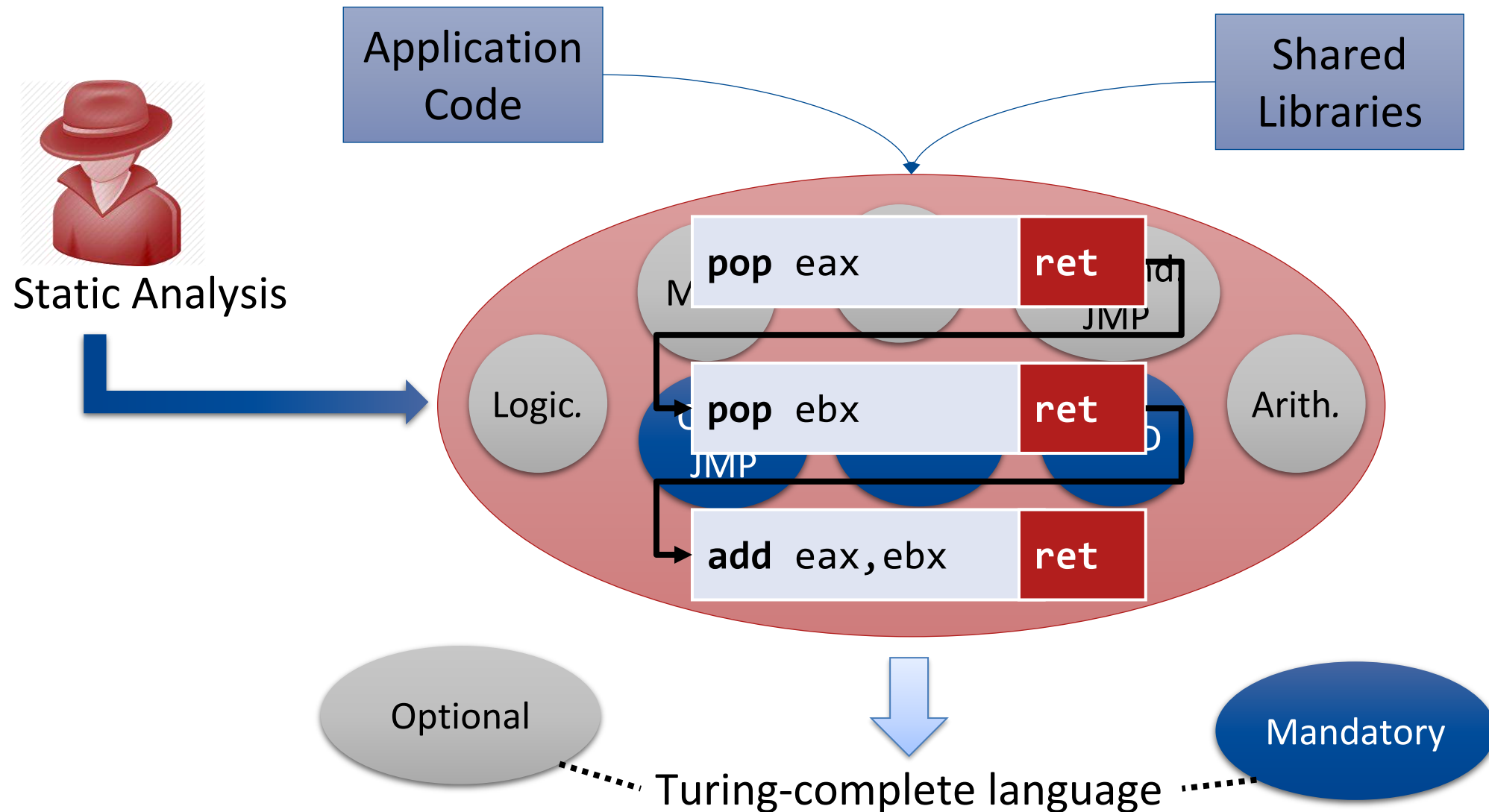
Attack Technique: Overview on x86



Memory Load Gadget on x86



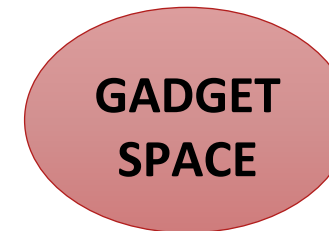
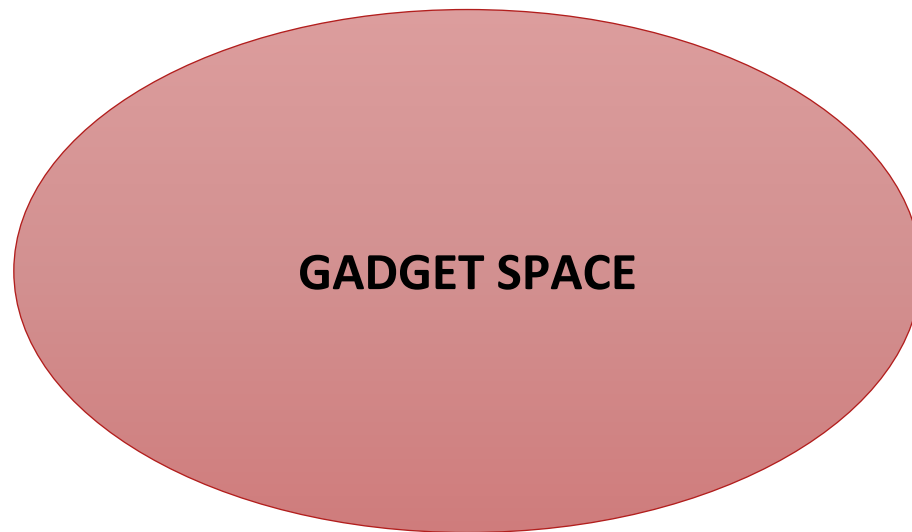
Code Base and Turing-Completeness



Gadget Space on Different Architectures

Architectures with no memory alignment, e.g., Intel x86

Architectures with memory alignment, e.g., SPARC, ARM



B8 13 00 00 00 E9 C3 F8 FF FF



Intended Code

```
mov eax,0x13  
jmp 3aae9
```

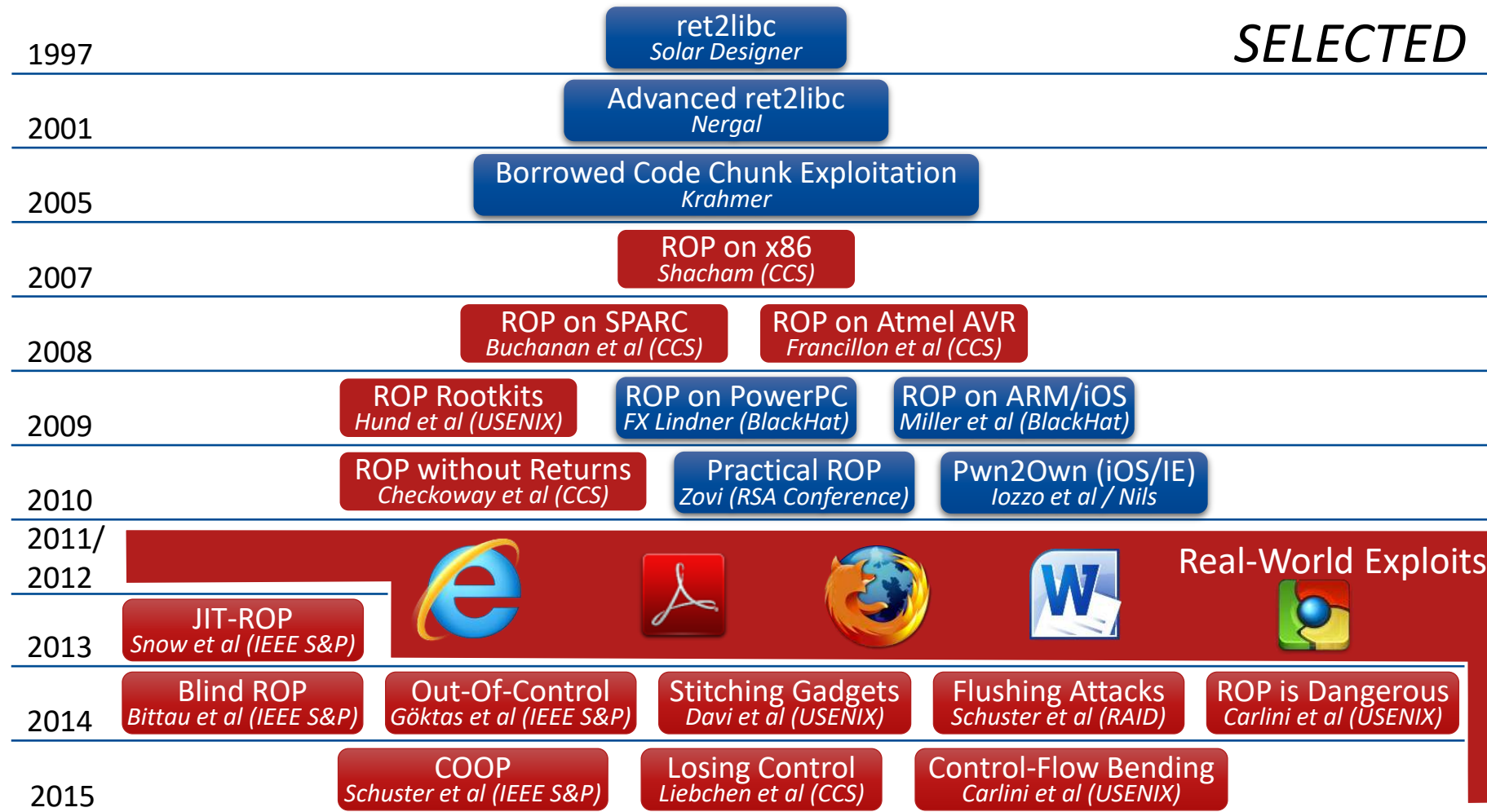
00 00 00 E9 C3



Unintended Code

```
add eax,[a1]  
add cl,ch  
ret
```

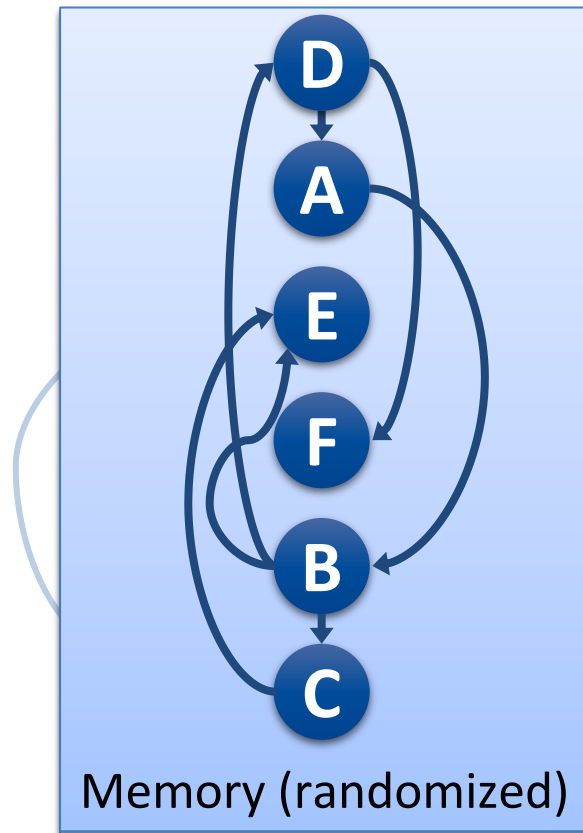
Research on Code-Reuse Attacks



Probabilistic vs Enforcement-Based Defense Approach

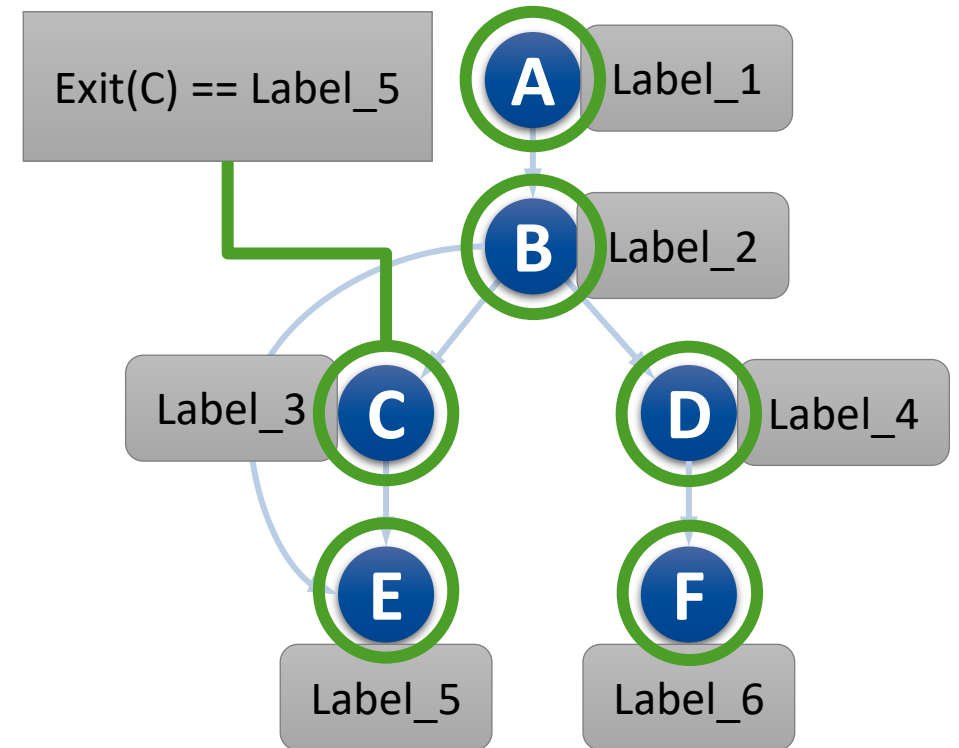
(Fine-grained) Code Randomization

[Cohen 1993 & Larsen et al., SoK IEEE S&P 2014]



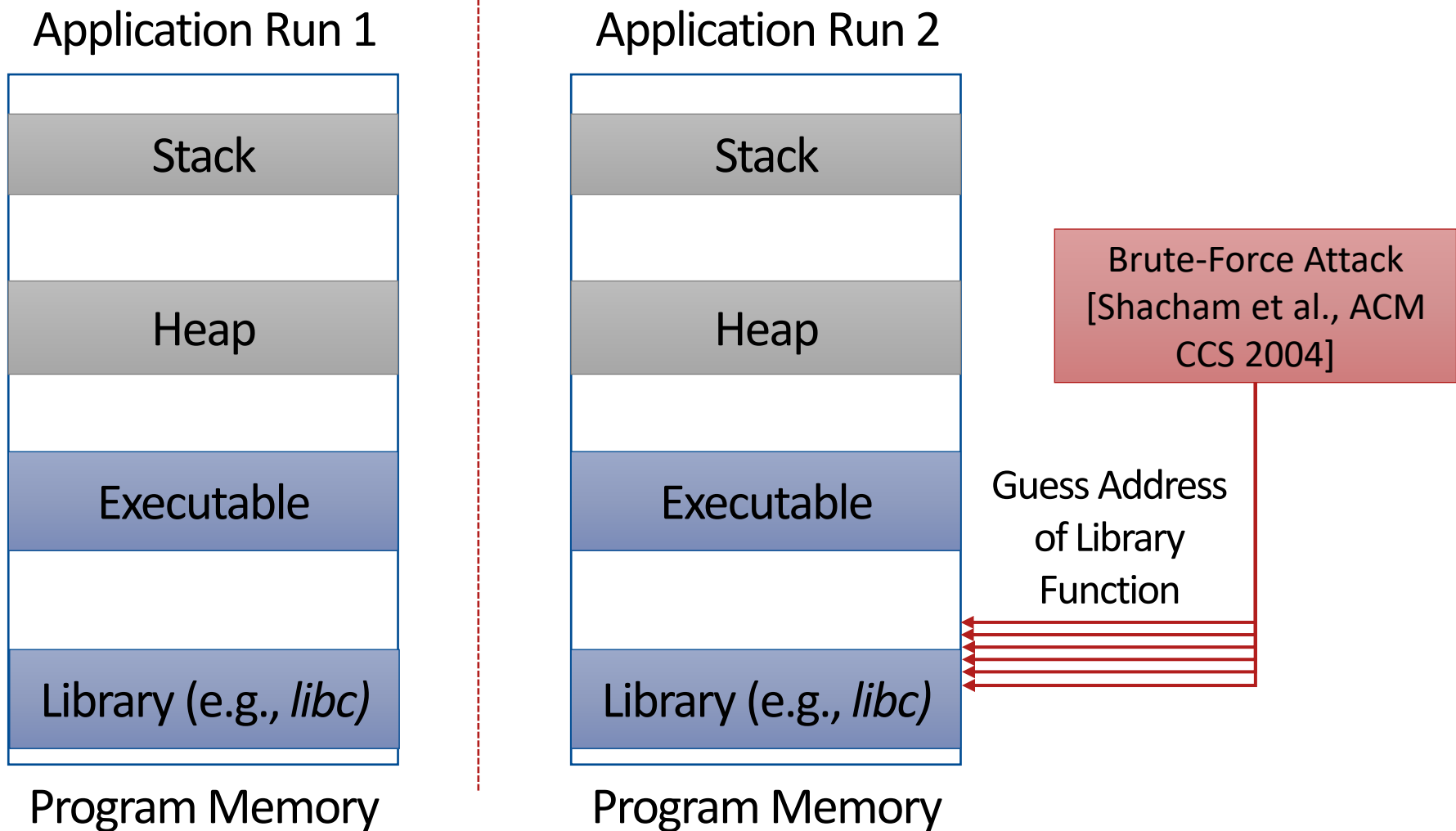
Control-Flow Integrity (CFI)

[Abadi et al., CCS 2005 & TISSEC 2009]

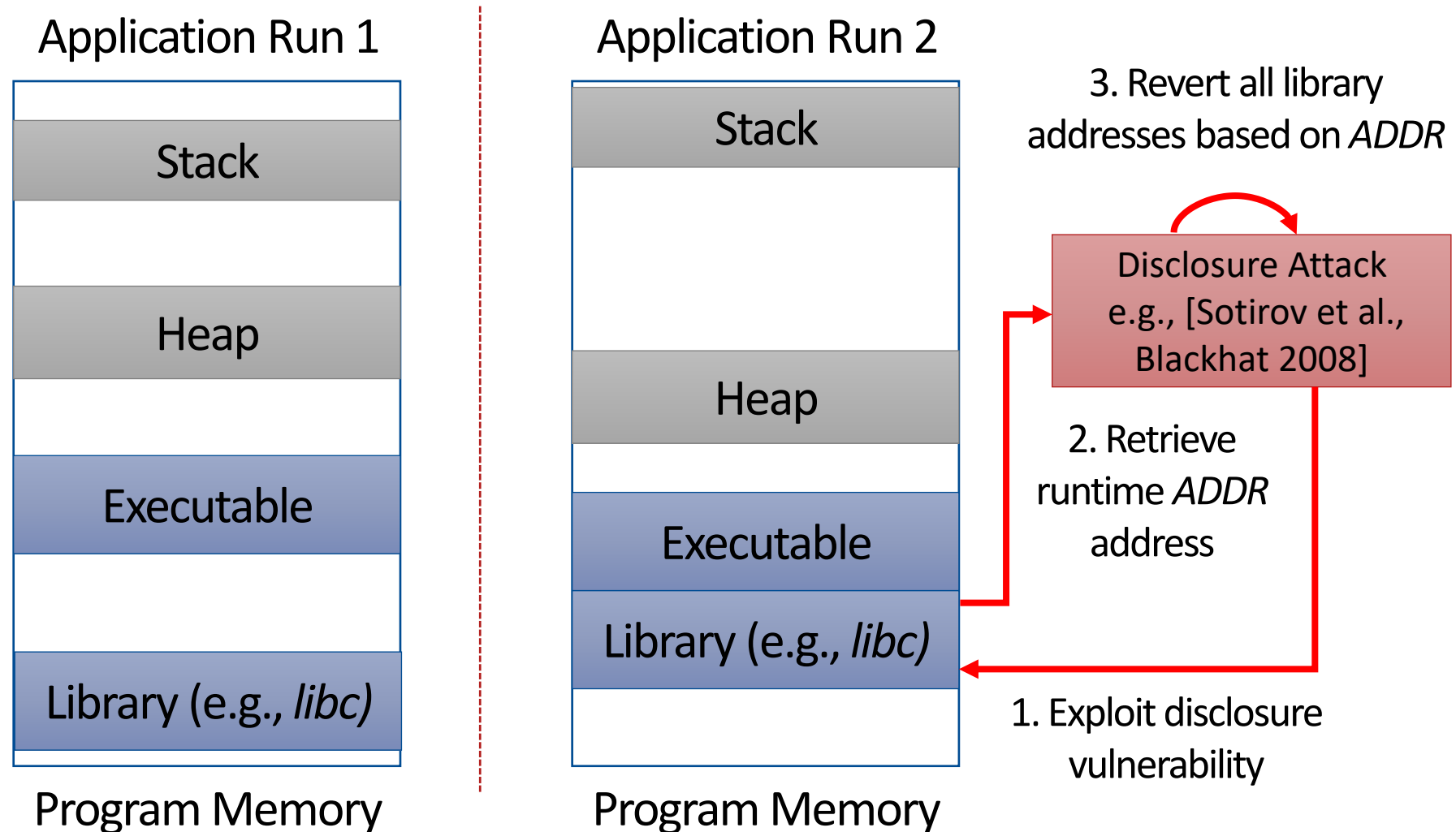


Basics of Code Randomization

ASLR randomizes the base address of code/data segments

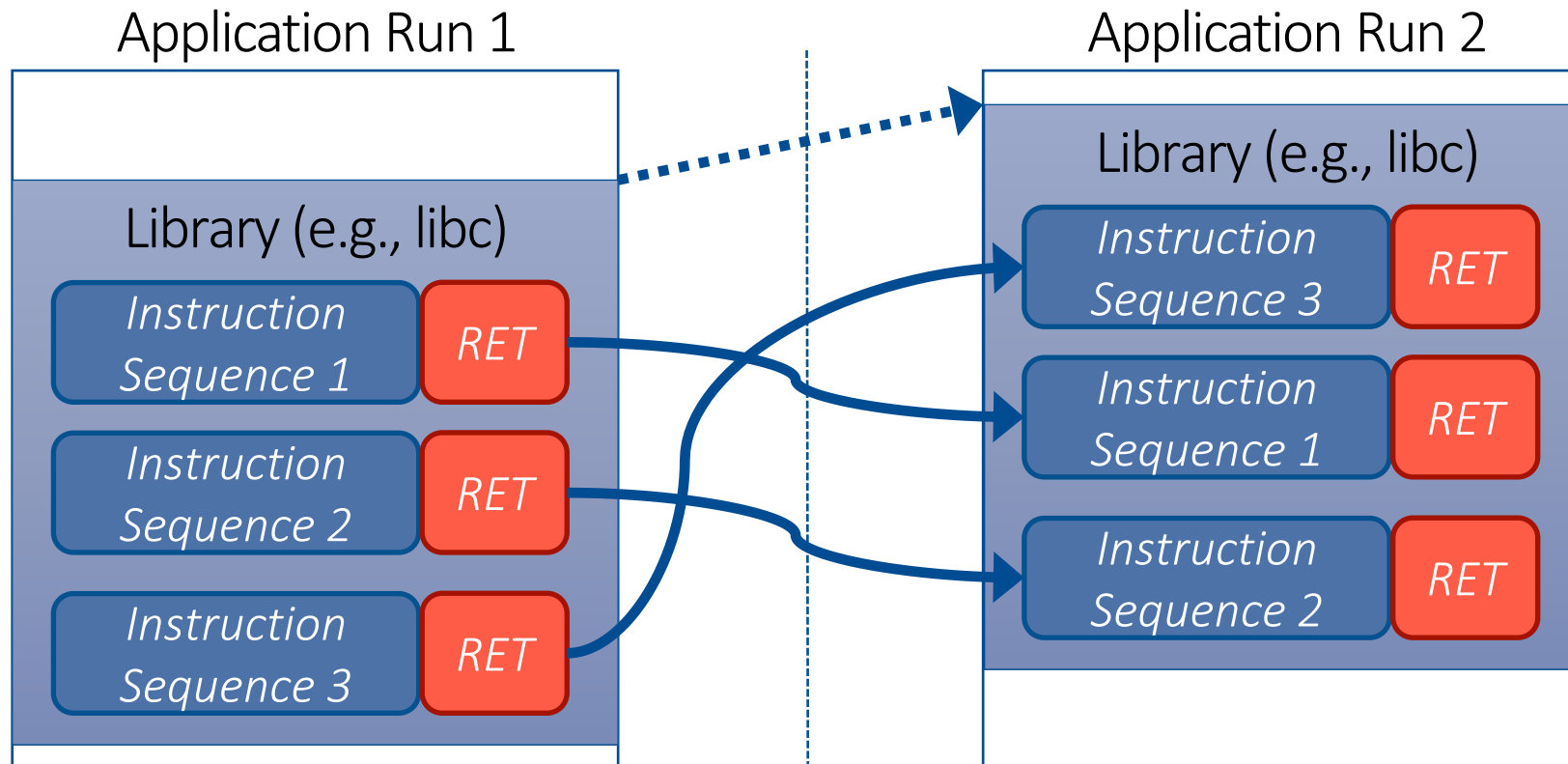


Memory Disclosure Attacks



Making exploitation harder with fine-grained code randomization

Fine-Grained ASLR



- **ORP** [Pappas et al., IEEE S&P 2012]: Instruction reordering/substitution within a BBL
- **ILR** [Hiser et al., IEEE S&P 2012]: Randomizing each instruction's location
- **STIR** [Wartell et al., ACM CCS 2012] & **XIFER** [Davi et al., AsiaCCS 2013]: Permutation of BBLs

Does Fine-Grained ASLR Provide a Viable Defense in the Long Run?



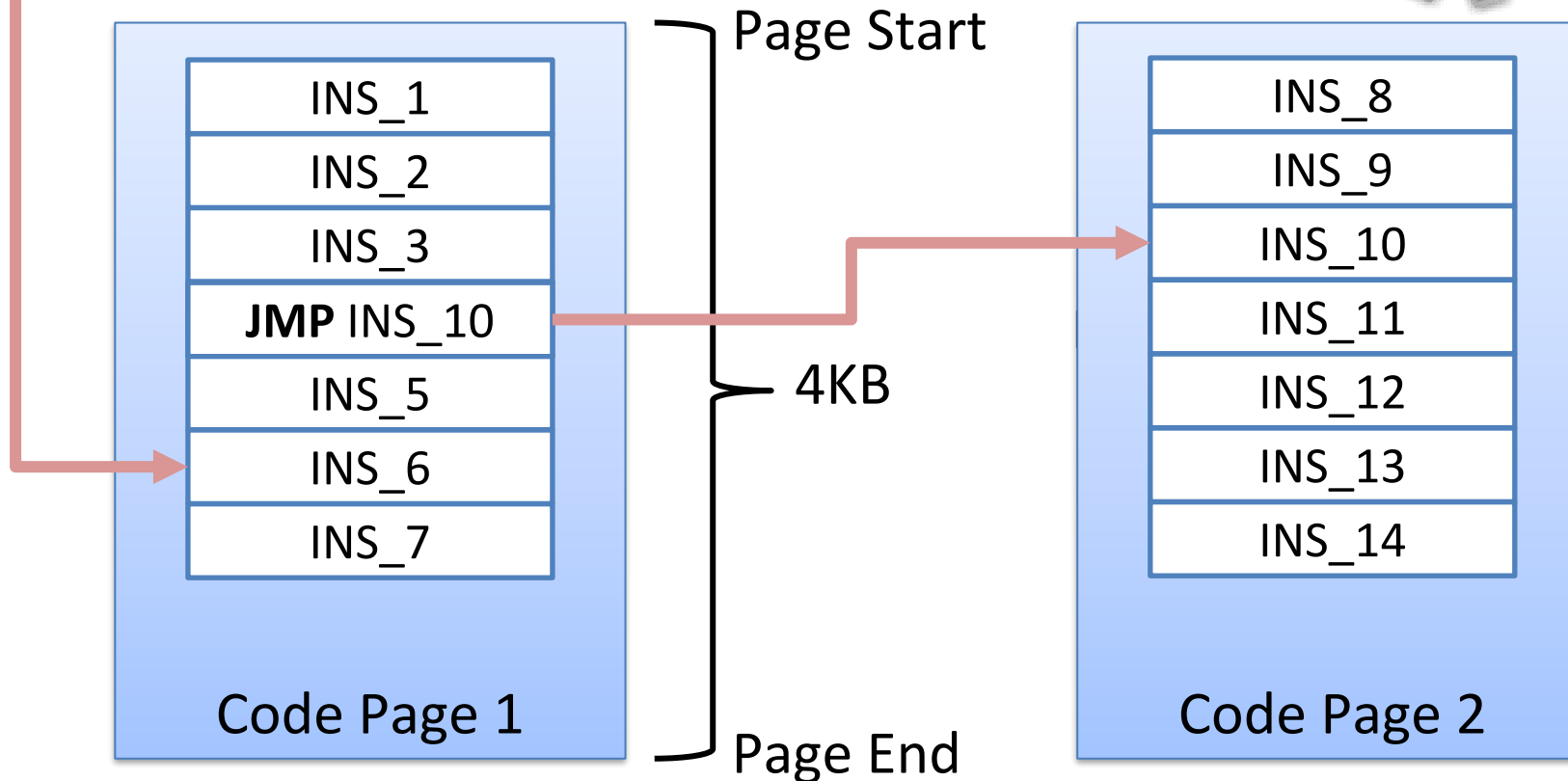
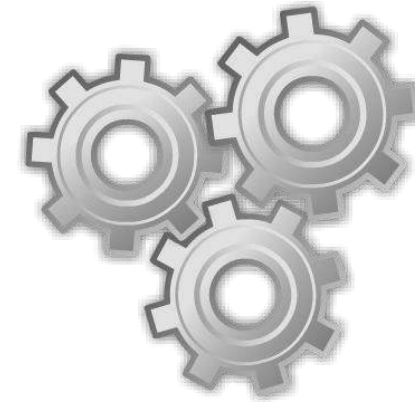
Just-In-Time Code Reuse: On the Effectiveness of Fine-Grained
Address Space Layout Randomization

[Kevin Snow, Lucas Davi, Alexandra Dmitrienko, Christopher Liebchen, Fabian
Monrose, Ahmad-Reza Sadeghi, Best Student Paper at IEEE S&P 2013]



Code Pointer

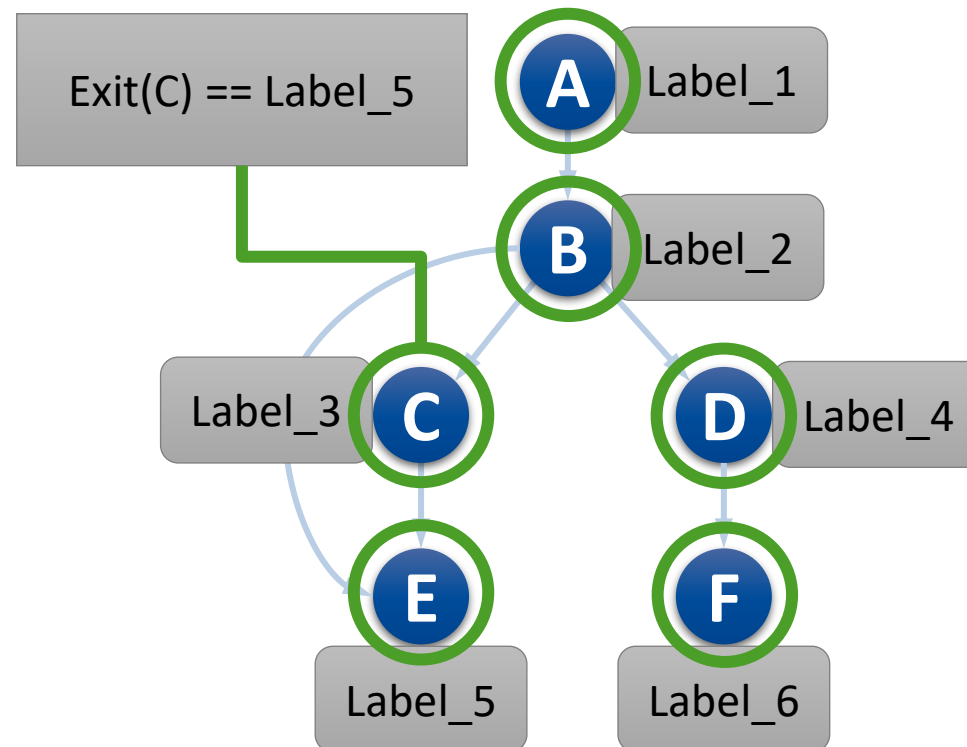
Scripting Engine



Making exploitation harder with control-flow integrity

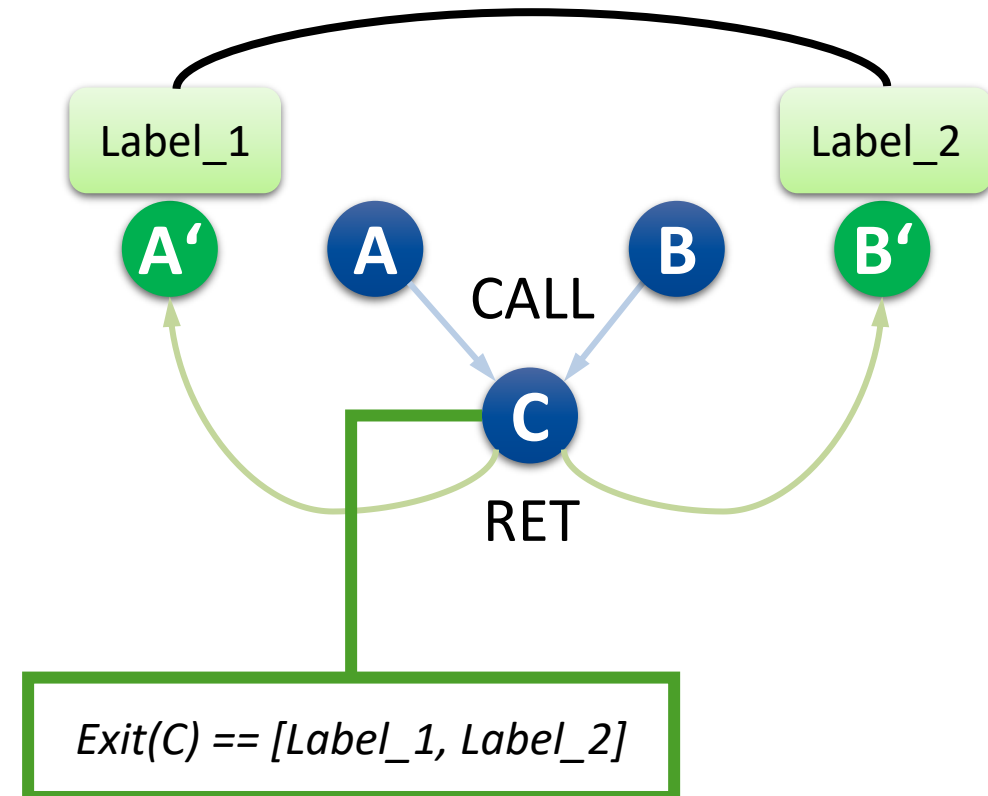
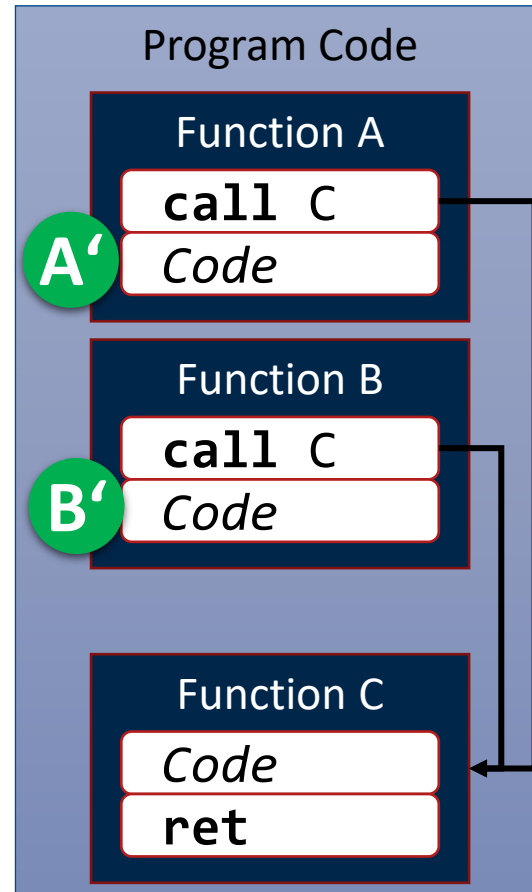
Control-Flow Integrity (CFI)

[Abadi et al., CCS 2005 &
TISSEC 2009]



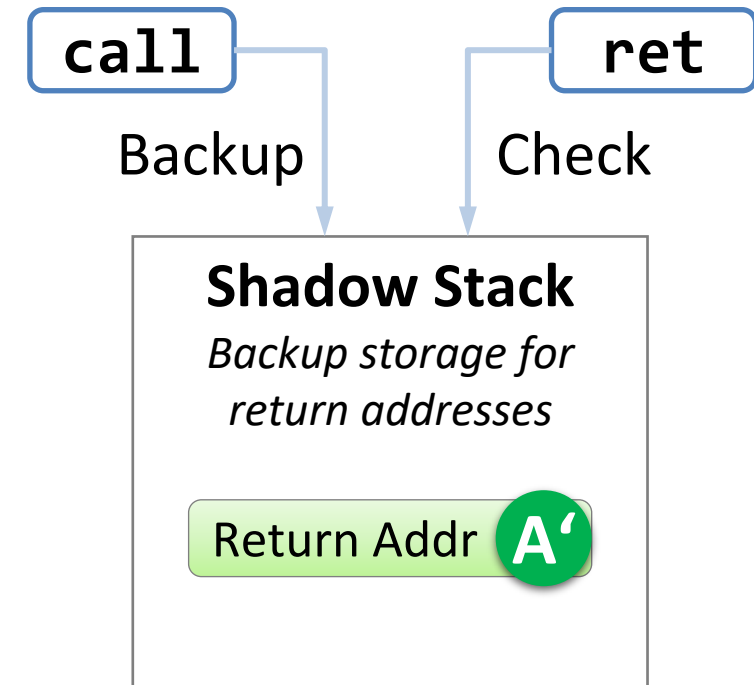
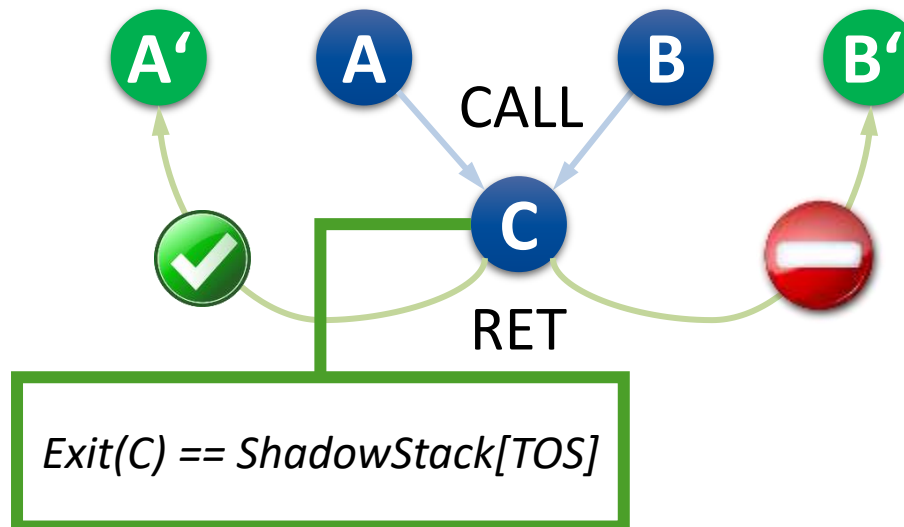
Label Problem for Returns

- Static CFI label checking leads to coarse-grained protection for returns

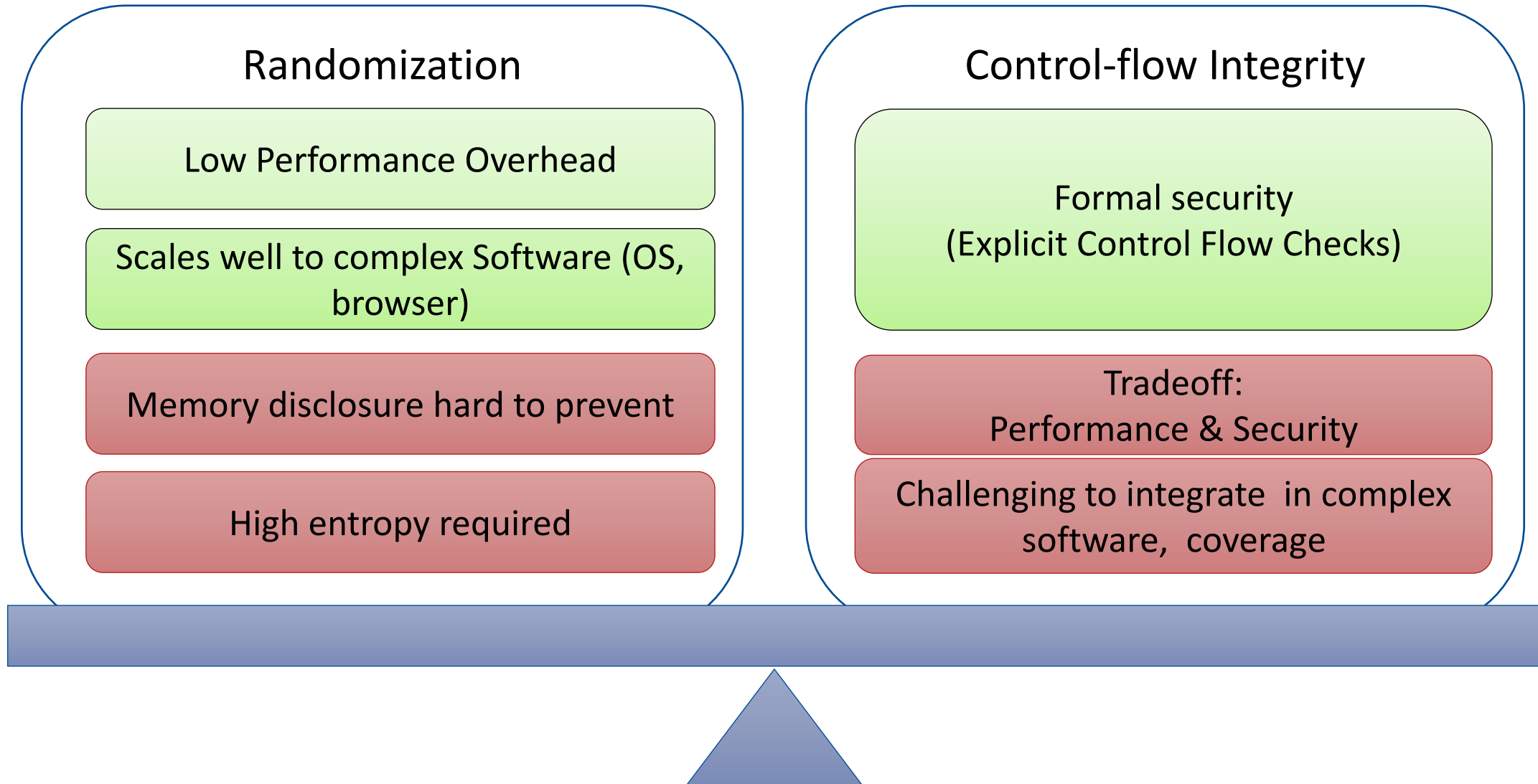


Shadow Stack / Return Address Stack

- ♦ **Shadow stack** allows for fine-grained return address protection but incurs higher overhead



Randomization vs. CFI



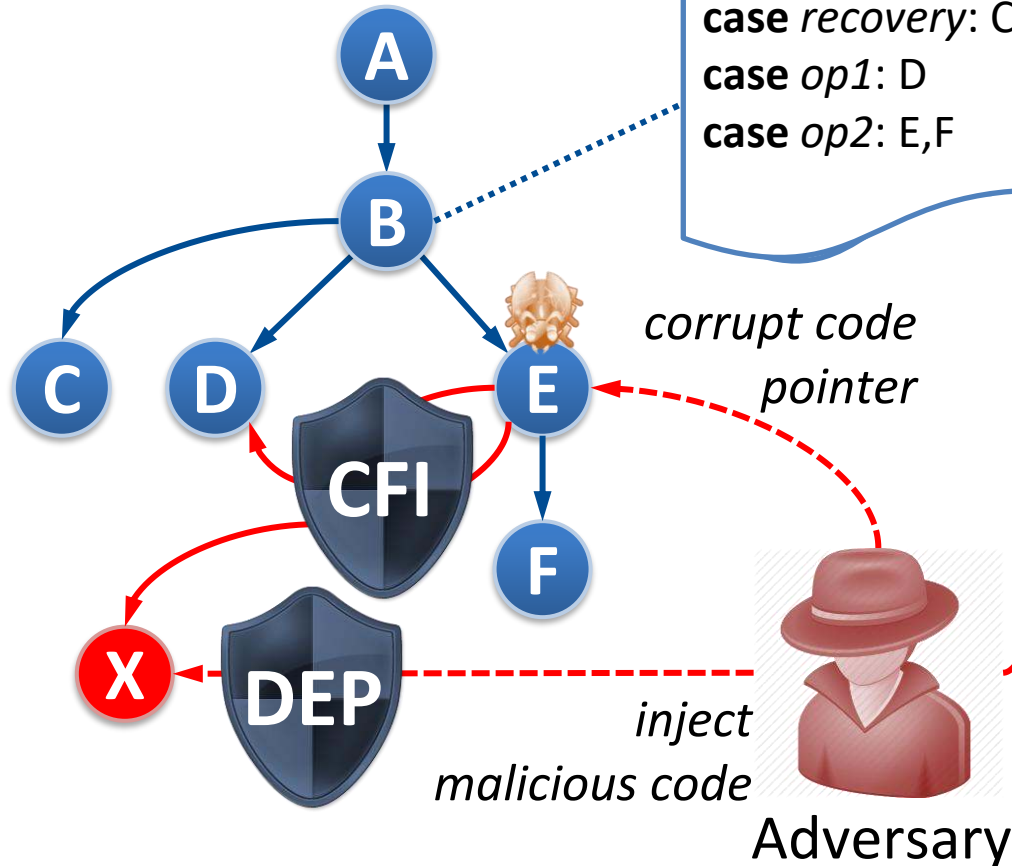
Data-Oriented Exploits

Problem Space

Control-Flow Attack

[Shacham, ACM CCS 2007]

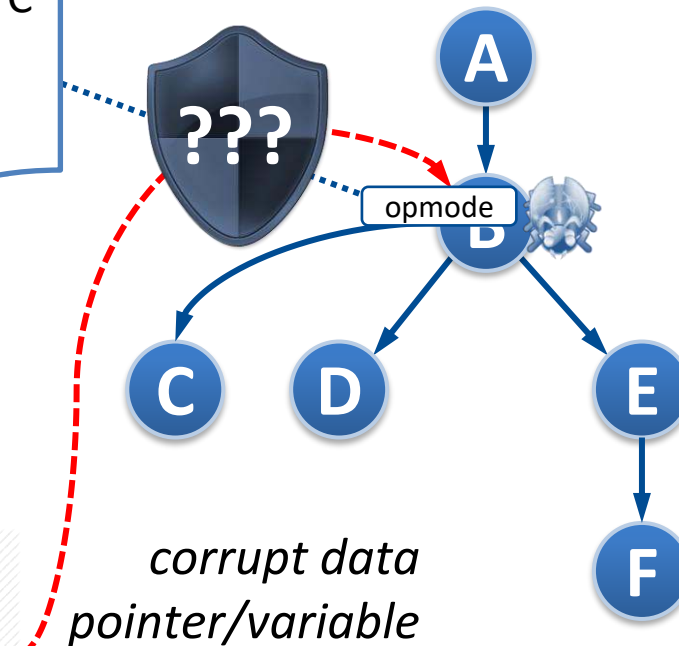
[Schuster et al., IEEE S&P 2015]



Non-Control-Data Attack

[Chen et al., USENIX Sec. 2005]

[Hu et al., USENIX Sec. 2015]



--- Memory write
--- Program flow

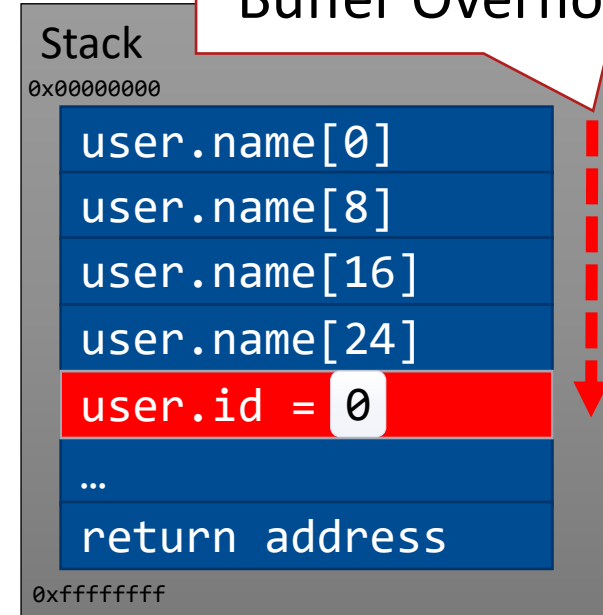
Data-Only Exploit on Example Code

```
int main() {  
    /* ... */  
    update_name(&user);  
    /* ... */  
    admin_access(&user);  
    /* ... */  
}
```

```
void update_name(user_t* user) {  
    fgets(user->name, 128, stdin);  
}
```

```
void admin_access(user_t* user) {  
    if (user->id == 0) {  
        admin_menu();  
    } else {  
        error();  
    }  
}
```

Buffer Overflow

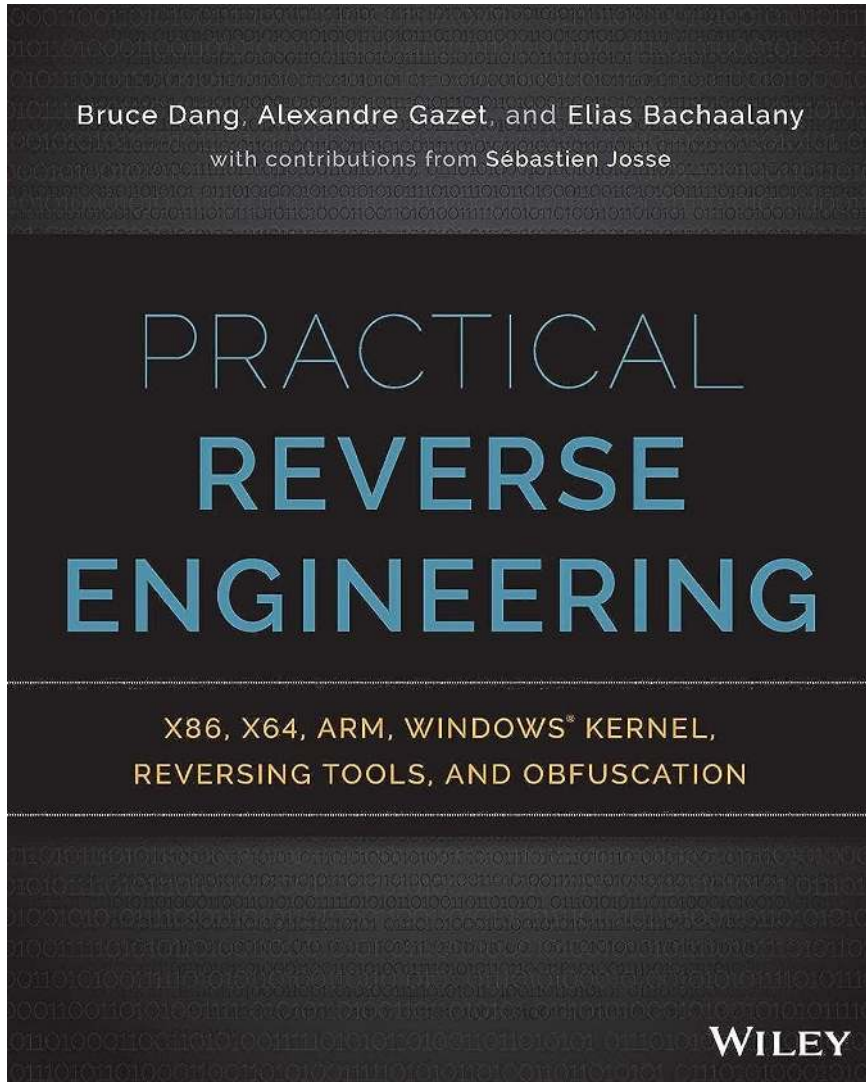


Same effect –
control-data left
untouched

References for Starting Research on Software Exploits

- L. Davi: Code-Reuse Attacks and Defenses [[Link](#)] PhD Thesis at TU Darmstadt 2016
- L. Szekeres, M. Payer, T. Wei, D. Song: SoK: Eternal War in Memory [[Link](#)] IEEE Security & Privacy 2013

References



In addition, check the Intel Homepage for detailed descriptions of assembler instructions and more:

<https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>

The full instruction reference can be found here:

<https://cdrdv2.intel.com/v1/dl/getContent/671110>

Vorlesung Cybersicherheit, Sommersemester 2025

Vielen Dank für Ihre Aufmerksamkeit

Prof. Dr.-Ing. Lucas Vincenzo Davi
Fakultät für Informatik
Arbeitsgruppe Systemsicherheit
Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi